Análisis y Diseño de Algoritmos I

Valeria Bengolea
Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

Técnicas de Diseño: Fuerza Bruta

Notas basadas en el material preparado por el Dr. Nazareno Aguirre

Fuerza Bruta

Es una técnica directa, que generalmente está basada en la definición del problema, y de los conceptos involucrados en la definición.

Ejemplos:

- Computar a^n ($a > 0$, n un entero no negativo)
- Computar n!
- Buscar la clave de un valor dado en una lista
- Calcular Máximo Común divisor

Ejemplo:
Ordenamiento por Fuerza Bruta

Dada una lista de n elementos comparables, ordenarlos en orden creciente.

Ejemplo: Ordenamiento por Fuerza Bruta

Ejemplo: Ordenamiento por Fuerza Bruta

Caso extremo: Calcular permutaciones del arreglo de entrada, hasta conseguir una permutación que esté ordenada.

Ejemplo: Ordenamiento por Fuerza Bruta

Caso extremo: Calcular permutaciones del arreglo de entrada, hasta conseguir una permutación que esté ordenada.

Ejemplo:

Ejemplo: Ordenamiento por Fuerza Bruta

Caso extremo: Calcular permutaciones del arreglo de entrada, hasta conseguir una permutación que esté ordenada.

Ejemplo: `2 1 3`

Ejemplo: Ordenamiento por Fuerza Bruta

Caso extremo: Calcular permutaciones del arreglo de entrada, hasta conseguir una permutación que esté ordenada.

Ejemplo: `2 1 3`

permutaciones

Ejemplo: Ordenamiento por Fuerza Bruta

Caso extremo: Calcular permutaciones del arreglo de entrada, hasta conseguir una permutación que esté ordenada.

Ejemplo: 2 1 3

permutaciones

2 1 3 2 3 1 3 2 1

3 1 2 1 3 2



ordenada

Ejemplo: Ordenamiento por Fuerza Bruta

Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

Ejemplo:

Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

Ejemplo: `7 3 2 5`

Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

Ejemplo: `7 3 2 5`

iteraciones

Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

Ejemplo: `7 3 2 5`

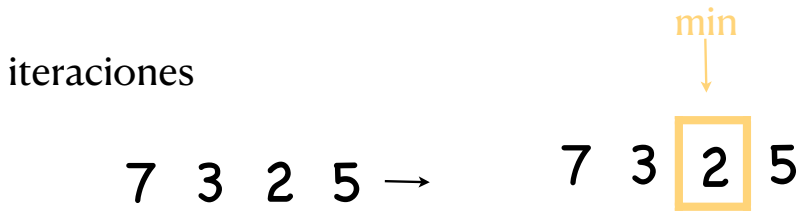
iteraciones

`7 3 2 5`

Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

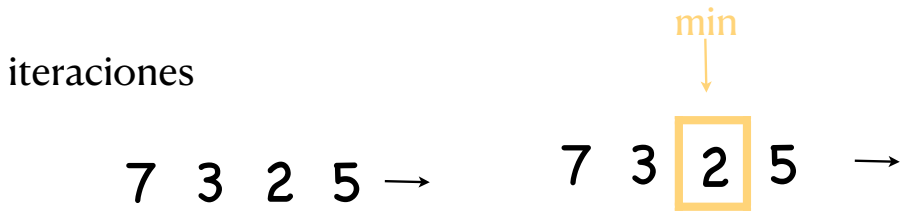
Ejemplo: 7 3 2 5



Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

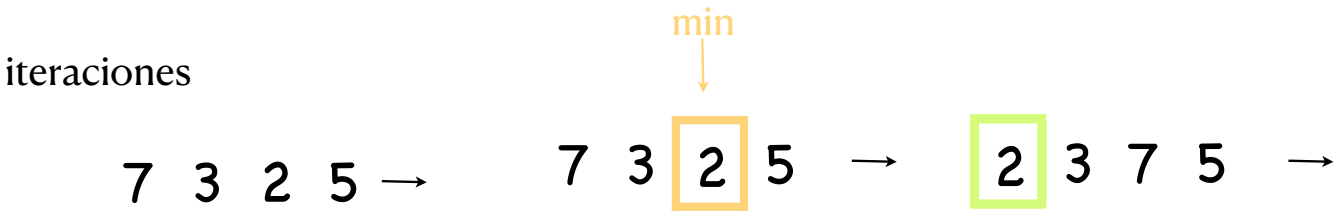
Ejemplo: 7 3 2 5



Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

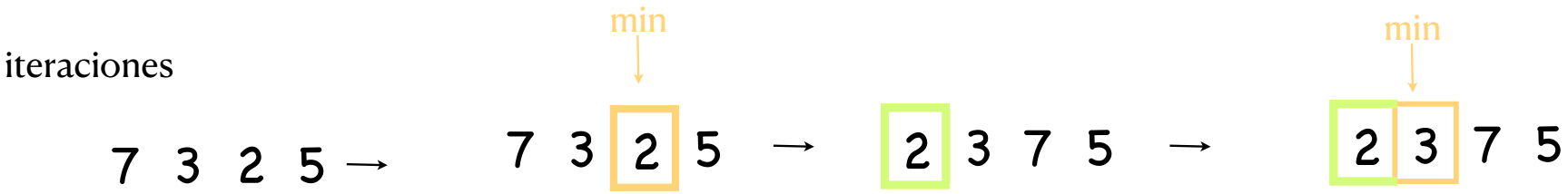
Ejemplo: 7 3 2 5



Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

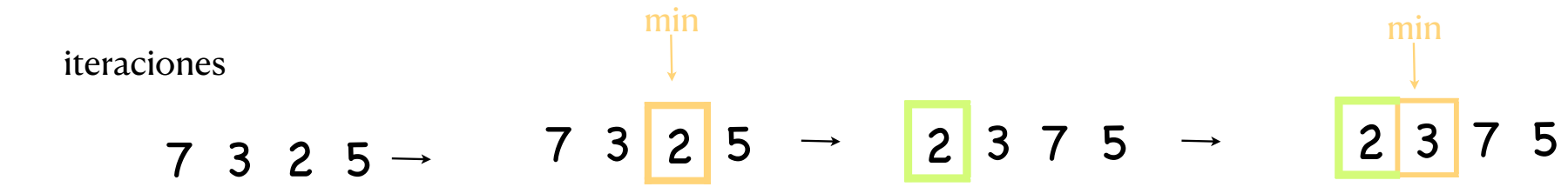
Ejemplo: 7 3 2 5



Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

Ejemplo: 7 3 2 5

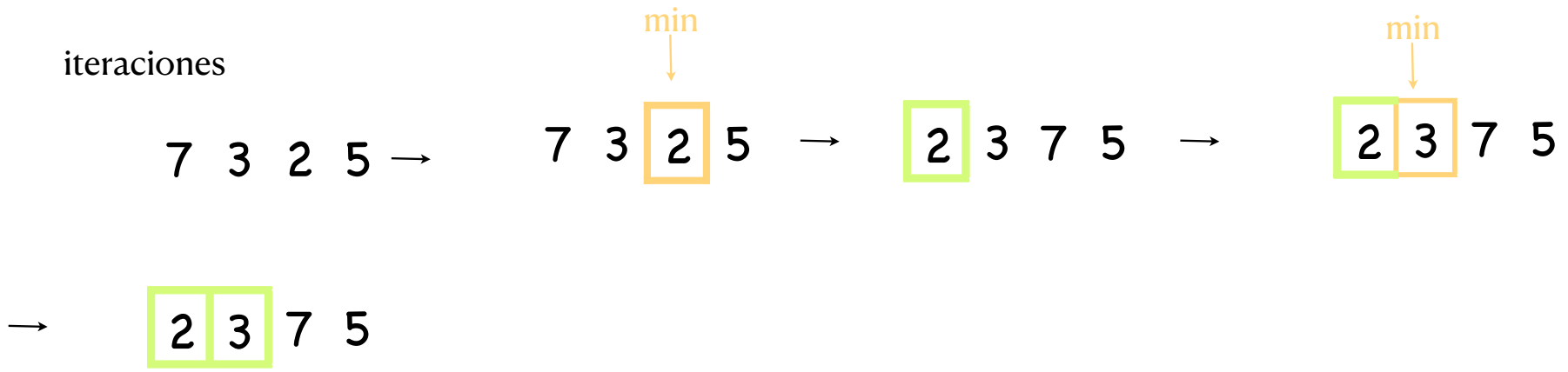


→

Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

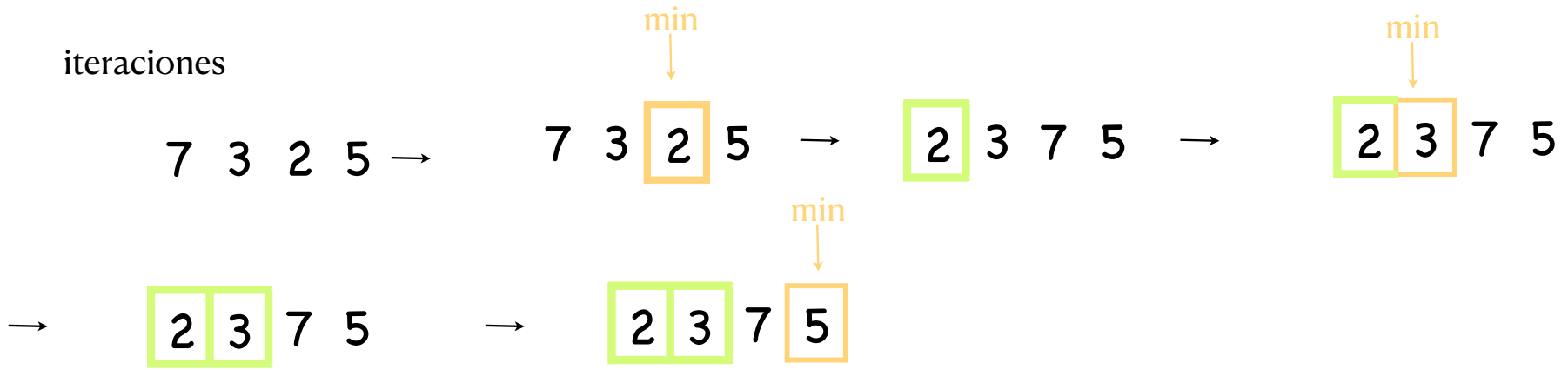
Ejemplo: 7 3 2 5



Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

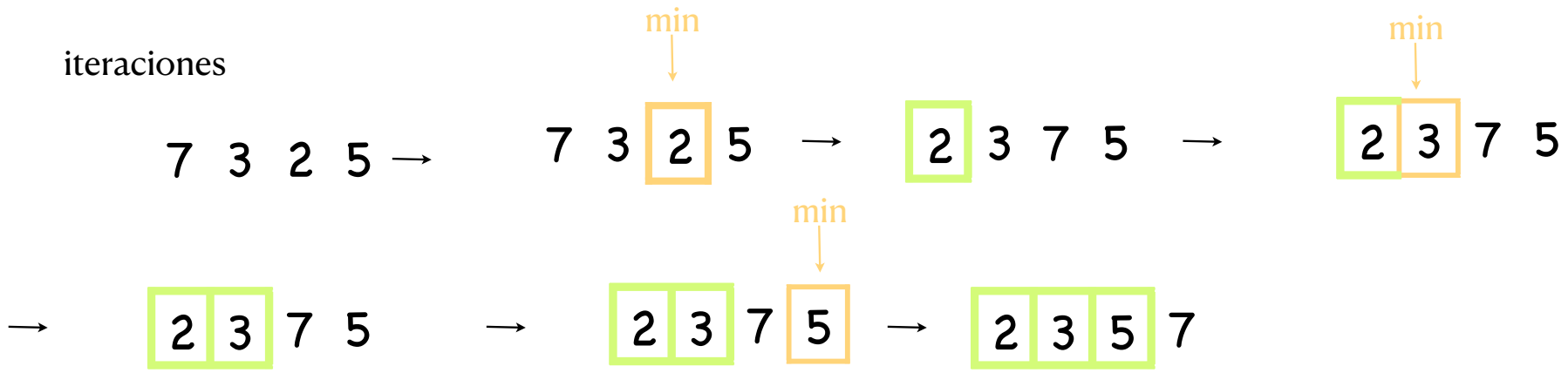
Ejemplo: 7 3 2 5



Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

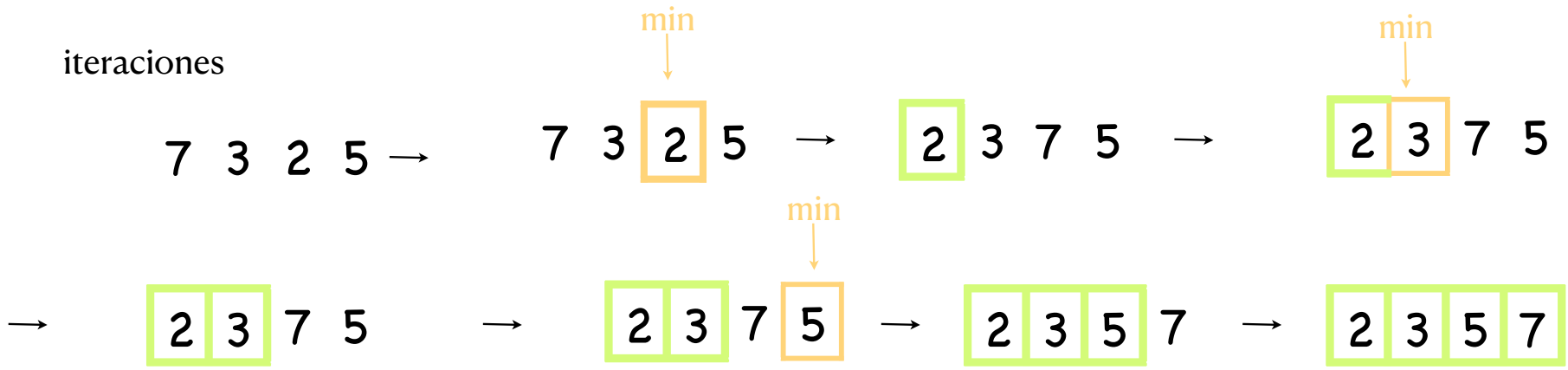
Ejemplo: 7 3 2 5



Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

Ejemplo: 7 3 2 5

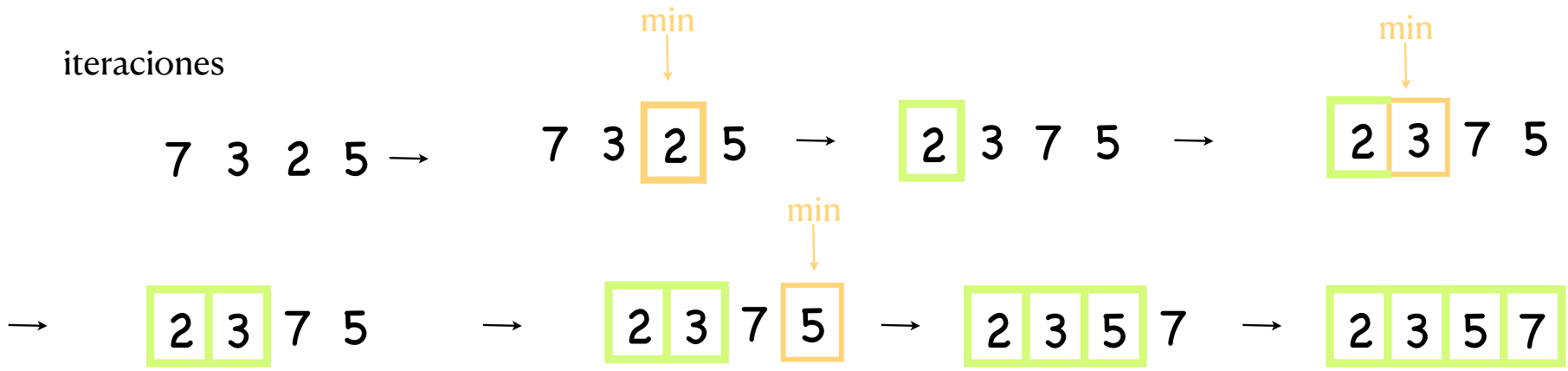


Ejemplo: Ordenamiento por Fuerza Bruta

Alternativa: Visitar el arreglo hasta encontrar el elemento mas chico, e intercambiarlo por el primer elemento. Luego, comenzando con el segundo elemento, buscar el elemento más chico a su derecha, e intercambiarlo con el segundo elemento. En general, en la iteración i (0<=i<=n-2), buscar el elemento más chico en A[i+1..n-1], e intercambiarlo con A[i].

Ejemplo: 7 3 2 5

Selection Sort



Análisis de Selection Sort

```
Algoritmo SelectionSort(A[0..n-1])
  // Ordena un arreglo mediante el proceso de selección.
  // Entrada: un arreglo A[0..n-1] de elementos comparables
  // Salida: el arreglo A[0..n-1], ordenado de manera ascendente.
  for i ← 0 to n-2 do
    min ← i
    for j ← i+1 to n-1 do
      if A[j]<A[min] then min ← j
    swap(A[i],A[min])
```

Análisis de Selection Sort

```
Algoritmo SelectionSort(A[0..n-1])
  // Ordena un arreglo mediante el proceso de selección.
  // Entrada: un arreglo A[0..n-1] de elementos comparables
  // Salida: el arreglo A[0..n-1], ordenado de manera ascendente.
  for i ← 0 to n-2 do
    min ← i
    for j ← i+1 to n-1 do
      if A[j]<A[min] then min ← j
    swap(A[i],A[min])
```

- ¿Cuál es la eficiencia en tiempo y espacio de este algoritmo?
- ¿Es este algoritmo estable?
- ¿Es este algoritmo correcto?

Análisis de Selection Sort

Análisis de Selection Sort

Estable

Análisis de Selection Sort

Estable

4 16 4 2

Análisis de Selection Sort

Estable

4 16 4 2

iteraciones

Análisis de Selection Sort

Estable

4 16 4 2

iteraciones

4 16 4 2

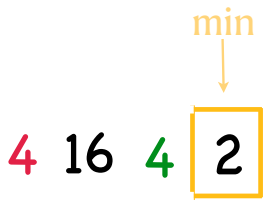
Análisis de Selection Sort

Estable

4 16 4 2

iteraciones

4 16 4 2 →



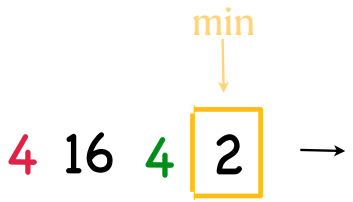
Análisis de Selection Sort

Estable

4 16 4 2

iteraciones

4 16 4 2 →

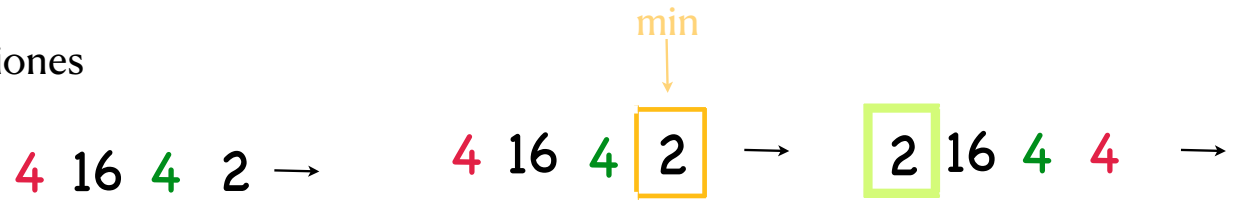


Análisis de Selection Sort

Estable

4 16 4 2

iteraciones

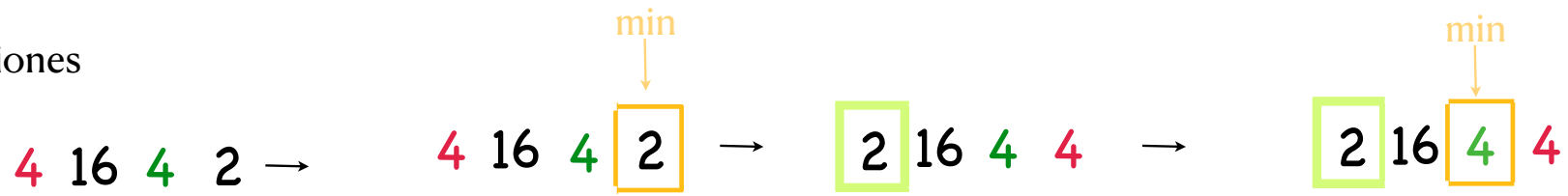


Análisis de Selection Sort

Estable

4 16 4 2

iteraciones

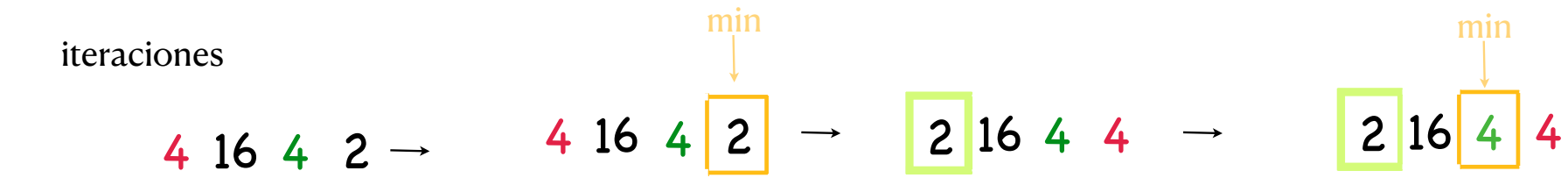


Análisis de Selection Sort

Estable

4 16 4 2

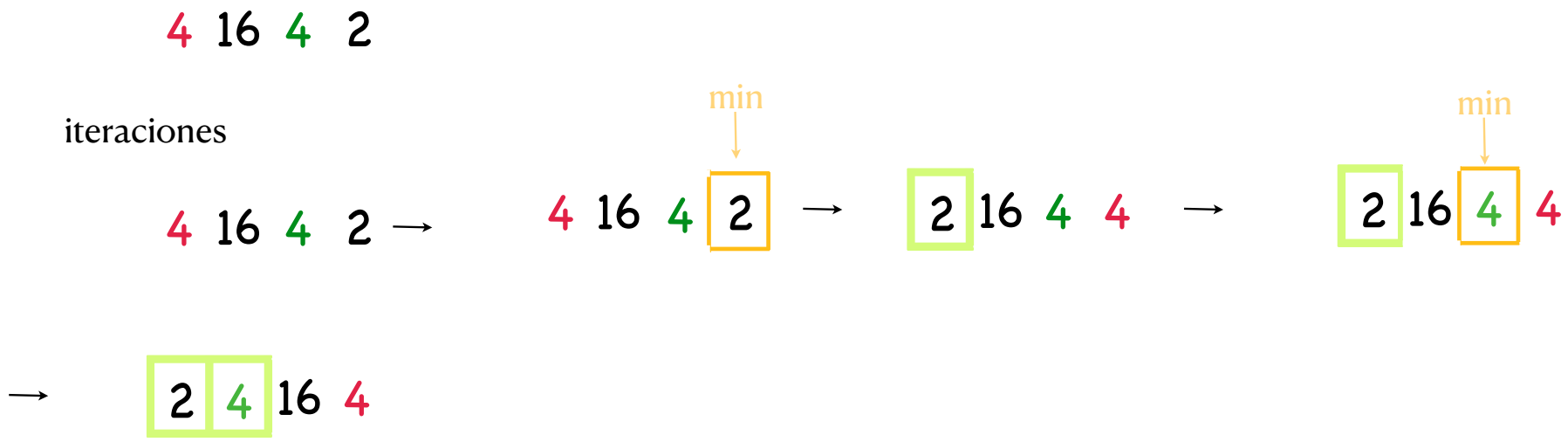
iteraciones



→

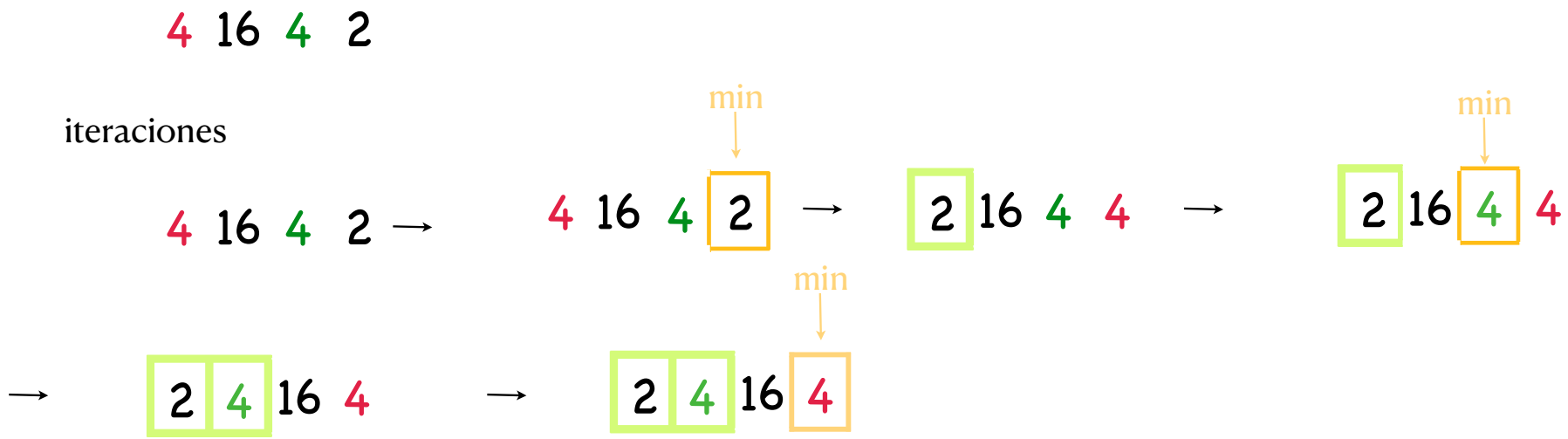
Análisis de Selection Sort

Estable



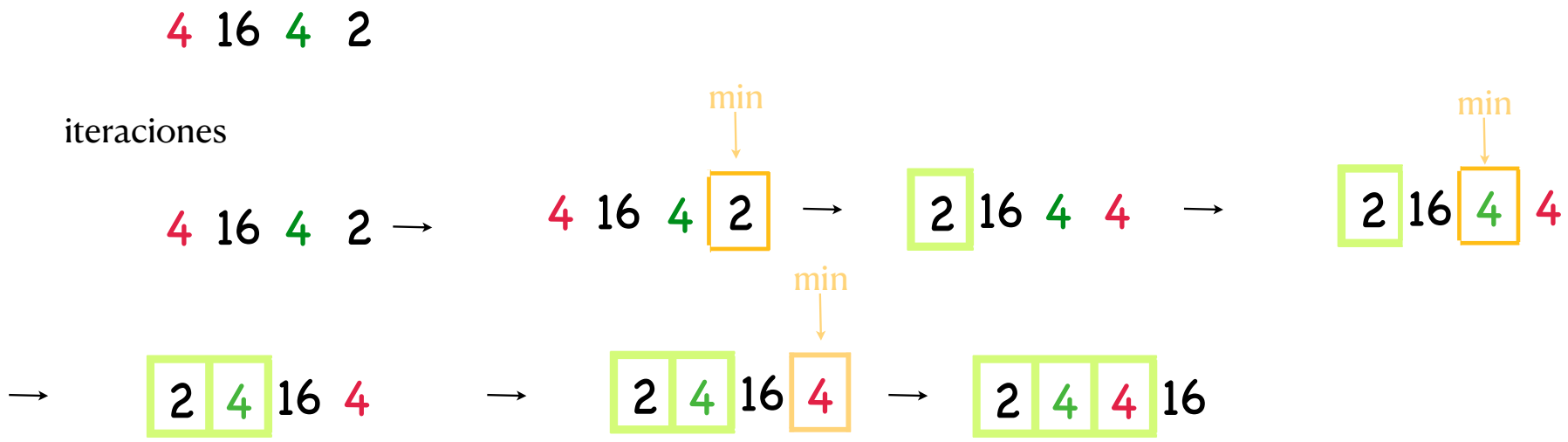
Análisis de Selection Sort

Estable



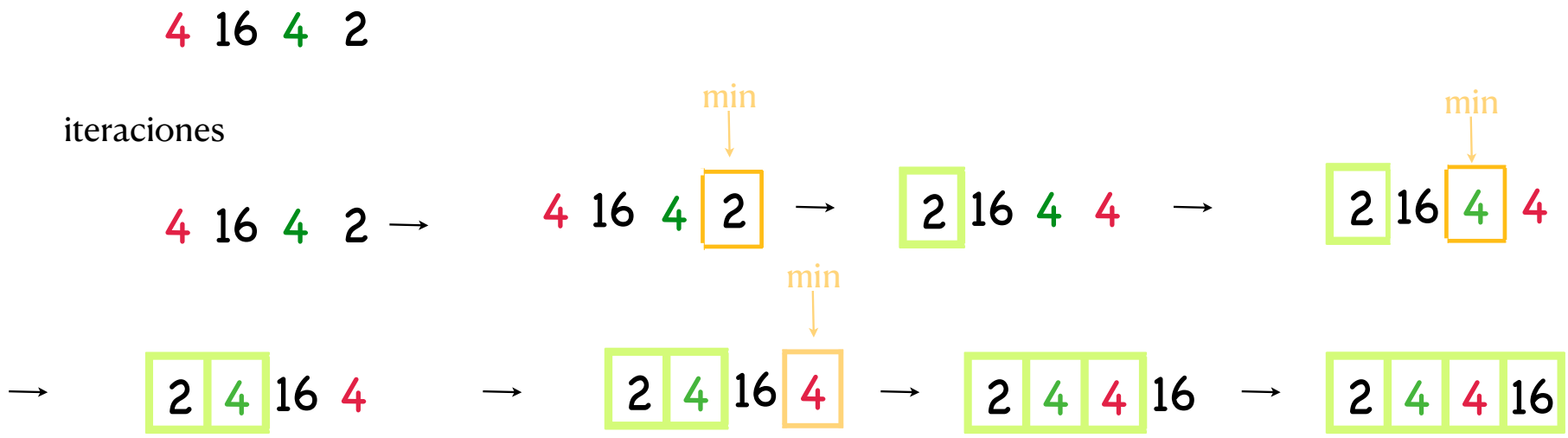
Análisis de Selection Sort

Estable



Análisis de Selection Sort

Estable



Análisis de tiempo de Ejecución Selection Sort

Operación de interés: comparación de claves

Todas las entradas de tamaño producen el mismo tiempo de ejecución

```
Algoritmo SelectionSort(A[0..n-1])
  for i ← 0 to n-2 do
    min ← i
    for j ← i+1 to n-1 do
      if A[j]<A[min] then min ← j
    swap(A[i],A[min])
```

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i) \quad == \frac{(n-1)n}{2}.$$

Análisis de tiempo de Ejecución Selection Sort

Operación de interés: comparación de claves

Todas las entradas de tamaño producen el mismo tiempo de ejecución

```
Algoritmo SelectionSort(A[0..n-1])
  for i ← 0 to n-2 do
    min ← i
    for j ← i+1 to n-1 do
      if A[j]<A[min] then min ← j
    swap(A[i],A[min])
```

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i) \quad == \frac{(n-1)n}{2}$$



Análisis de tiempo de Ejecución
Selection Sort

$$\sum_{i=0}^{n-2} (n - 1 - i)$$

Análisis de tiempo de Ejecución
Selection Sort

$$\sum_{i=0}^{n-2} (n-1-i)$$

(n-1-0) + (n-1-1) + (n-1-2) + ...(n-1-(n-2))
n-1 + n-2 + n-3 + ...+ 1

Análisis de tiempo de Ejecución
Selection Sort

$$\sum_{i=0}^{n-2} (n-1-i)$$

(n-1-0) + (n-1-1) + (n-1-2) + ...(n-1-(n-2))
n-1 + n-2 + n-3 + ...+ 1

$$= \sum_{i=1}^{n-1} (n-i)$$

Análisis de tiempo de Ejecución
Selection Sort

$$\sum_{i=0}^{n-2} (n-1-i)$$

(n-1-0) + (n-1-1) + (n-1-2) + ...(n-1-(n-2))
n-1 + n-2 + n-3 + ...+ 1

$$= \sum_{i=1}^{n-1} (n-i)$$

(n-1) +(n-2)+ (n-3) + ...+ n-(n-1)
n-1 + n-2 + n-3 + ...+ 1

Análisis de tiempo de Ejecución
Selection Sort

$$\sum_{i=0}^{n-2}(n-1-i)$$

(n-1-0) + (n-1-1) + (n-1-2) + ...(n-1-(n-2))
n-1 + n-2 + n-3 + ...+ 1

$$= \sum_{i=1}^{n-1}(n-i)$$

(n-1) +(n-2)+ (n-3) + ...+ n-(n-1)
n-1 + n-2 + n-3 + ...+ 1

$$= \sum_{i=1}^{n-1} i$$

Análisis de tiempo de Ejecución
Selection Sort

$$\sum_{i=0}^{n-2}(n-1-i)$$

(n-1-0) + (n-1-1) + (n-1-2) + ...(n-1-(n-2))
n-1 + n-2 + n-3 + ...+ 1

$$= \sum_{i=1}^{n-1}(n-i)$$

(n-1) +(n-2)+ (n-3) + ...+ n-(n-1)
n-1 + n-2 + n-3 + ...+ 1

$$= \sum_{i=1}^{n-1} i \quad = \frac{(n-1)(n-1+1)}{2}$$

Análisis de tiempo de Ejecución
Selection Sort

$$\sum_{i=0}^{n-2}(n-1-i)$$

(n-1-0) + (n-1-1) + (n-1-2) + ...(n-1-(n-2))
n-1 + n-2 + n-3 + ...+ 1

$$= \sum_{i=1}^{n-1}(n-i)$$

(n-1) +(n-2)+ (n-3) + ...+ n-(n-1)
n-1 + n-2 + n-3 + ...+ 1

$$= \sum_{i=1}^{n-1} i \quad = \frac{(n-1)(n-1+1)}{2} \quad = \frac{(n-1)n}{2}$$

Análisis de tiempo de Ejecución
Selection Sort

$$\sum_{i=0}^{n-2}(n-1-i)$$

(n-1-0) + (n-1-1) + (n-1-2) + ...(n-1-(n-2))
n-1 + n-2 + n-3 + ...+ 1

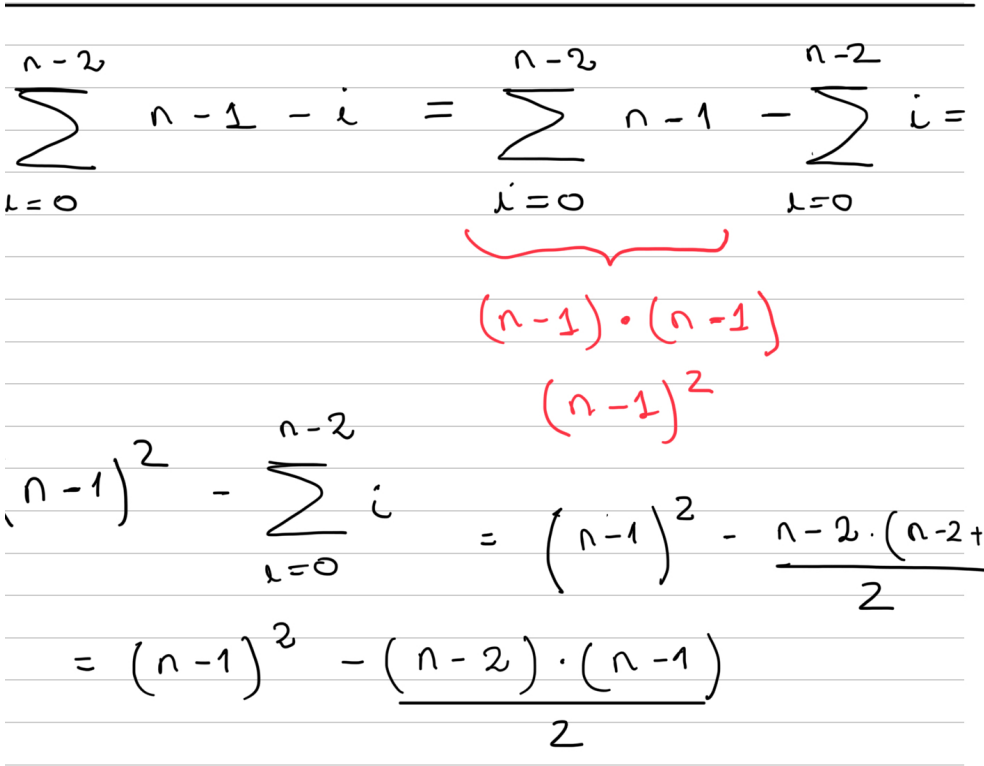
$$= \sum_{i=1}^{n-1}(n-i)$$

(n-1) +(n-2)+ (n-3) + ...+ n-(n-1)
n-1 + n-2 + n-3 + ...+ 1

$$= \sum_{i=1}^{n-1} i \quad = \frac{(n-1)(n-1+1)}{2} \quad = \frac{(n-1)n}{2} \quad \theta(n^2)$$

Análisis de tiempo de Ejecución Selection Sort

- Otra alternativa para el cálculo $\sum_{i=0}^{n-2}(n-1-i)$



Análisis de tiempo de Ejecución Selection Sort

- Otra alternativa para el cálculo $\sum_{i=0}^{n-2}(n-1-i)$

$$(n-1)(n-1) - (n-2)\cdot(n-1)\cdot\frac{1}{2}$$

$$(n-1)\left[(n-1) - \frac{(n-2)}{2}\right]$$

$$(n-1)\cdot\frac{2(n-1)-(n-2)}{2}$$

$$(n-1)\cdot\frac{2n-2-n+2}{2}$$

$$(n-1)\cdot\frac{2n-n}{2}$$

$$(n-1)\cdot\frac{n}{2}$$

$$= \frac{(n-1)n}{2}$$

Ejemplo: Ordenamiento por Fuerza Bruta

Comparar elementos adyacentes de la lista e intercambiarlos si no están en orden.

Ejemplo: Ordenamiento por Fuerza Bruta

Comparar elementos adyacentes de la lista e intercambiarlos si no están en orden.

ALGORITHM *BubbleSort*$(A[0..n-1])$

//Sorts a given array by bubble sort
//Input: An array $A[0..n-1]$ of orderable elements
//Output: Array $A[0..n-1]$ sorted in nondecreasing order
for $i \leftarrow 0$ **to** $n-2$ **do**
    for $j \leftarrow 0$ **to** $n-2-i$ **do**
        if $A[j+1] < A[j]$ swap $A[j]$ and $A[j+1]$

Análisis de tiempo de Ejecución BubbleSort

Comparar elementos adyacentes de la lista e intercambiarlos si no están en orden.

ALGORITHM *BubbleSort*$(A[0..n-1])$

```
//Sorts a given array by bubble sort
//Input: An array A[0..n − 1] of orderable elements
//Output: Array A[0..n − 1] sorted in nondecreasing order
for i ← 0 to n − 2 do
    for j ← 0 to n − 2 − i do
        if A[j + 1] < A[j] swap A[j] and A[j + 1]
```

Operación de interés: comparaciones de claves

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1$$

Necesito hacer distinción entre mejor/peor caso/Caso promedio?

Análisis de tiempo de Ejecución BubbleSort

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1$$

$$\sum_{i=0}^{n-2} n-2-i+1 = \sum_{i=0}^{n-2} n-1-i$$

$(n-1) \cdots (n-1-(n-2))$

$(n-1) \cdots 1$

$$\sum_{i=1}^{n-1} i$$

$$= \frac{(n-1)(n-1+1)}{2} = \frac{(n-1) \cdot n}{2}$$

Análisis de tiempo de Ejecución BubbleSort

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1$$

$$\theta(n^2)$$

$$\sum_{i=0}^{n-2} n-2-i+1 = \sum_{i=0}^{n-2} n-1-i$$

$(n-1) \cdots (n-1-(n-2))$

$(n-1) \cdots 1$

$$\sum_{i=1}^{n-1} i$$

$$= \frac{(n-1)(n-1+1)}{2} = \frac{(n-1) \cdot n}{2}$$

Ejemplo 2: Matching de Cadenas por Fuerza Bruta

Problema: Dadas un par de cadenas t y p, determinar si p es subcadena de t.

Ejemplo:

¿Es p: 001011 subcadena de t: 10010010111001100101111010 ?

Ejemplo 2:
Matching de Cadenas por Fuerza Bruta

Solución por fuerza bruta:

· Paso 1 Alinear el patrón al comienzo del texto

· Paso 2 Moverse de izquierda a derecha, comparando caracter por caracter del texto (**t**) con el patrón (**p**), hasta que todos los caracteres buscados se correspondan con el texto (**búsqueda exitosa**), o se encuentre una discrepancia

· Paso 3 Mientras el patrón no se encuentre, y el texto no se haya agotado, realinear el patrón una posición hacia la derecha, y volver al paso 2

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011

1 0 0 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 0 1 1 1 1 0 1 0

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011

1 0 0 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 0 1 1 1 1 0 1 0

0 0 1 0 1 1

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011

1 0 0 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 0 1 1 1 1 0 1 0

0 0 1 0 1 1

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011

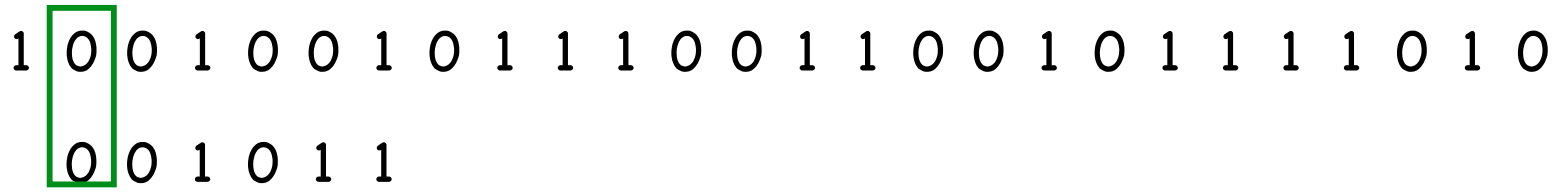
1 0 0 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 0 1 1 1 1 0 1 0

0 0 1 0 1 1

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

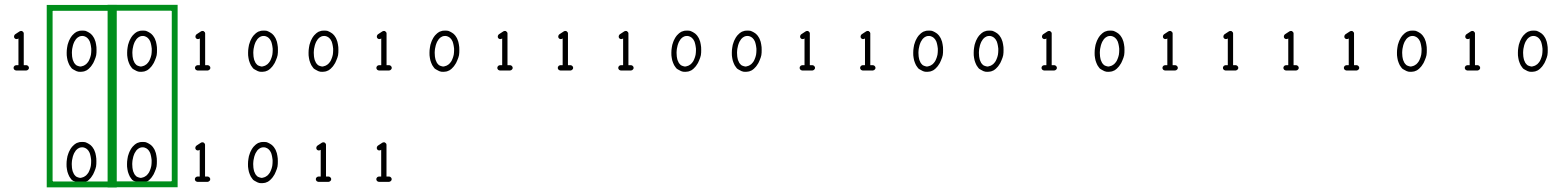
t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

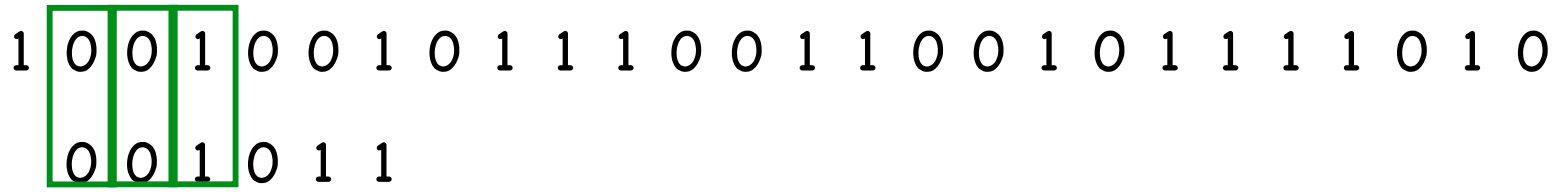
t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

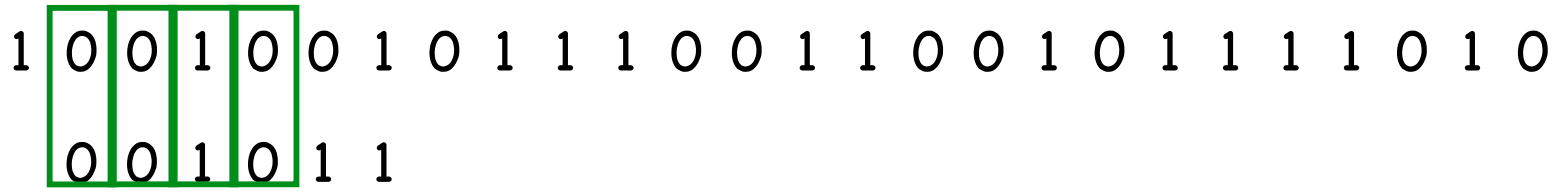
t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

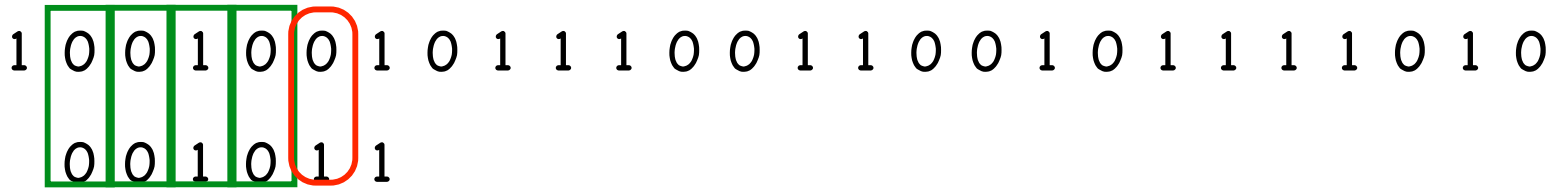
t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011

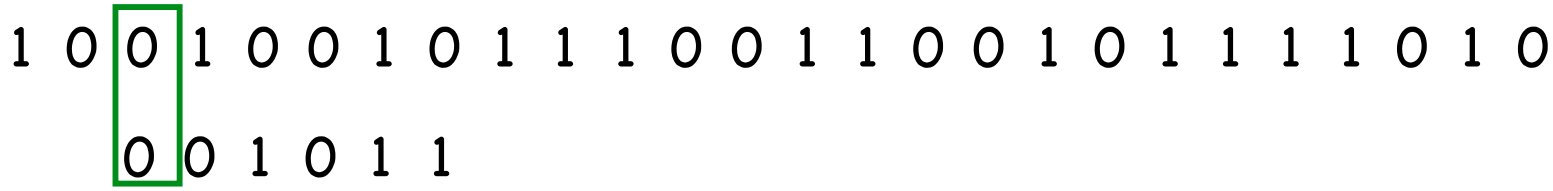
1 0 0 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 0 1 1 1 1 0 1 0

0 0 1 0 1 1

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

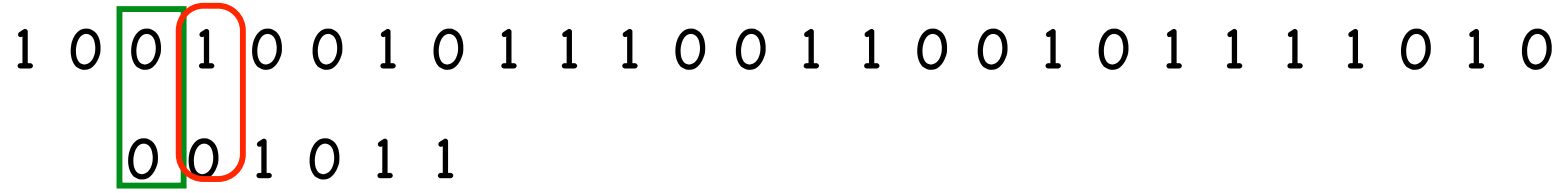
t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011

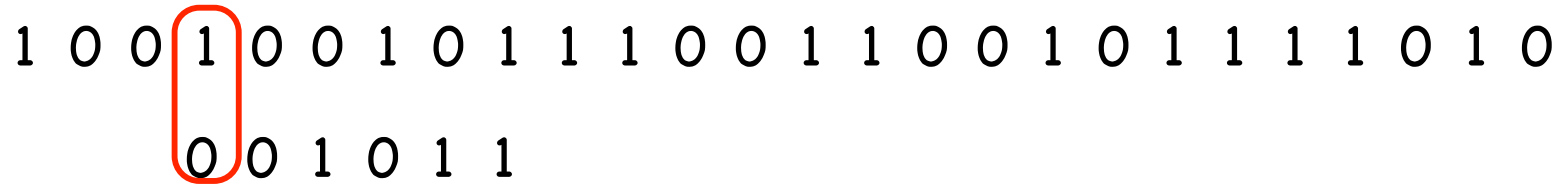
1 0 0 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 0 1 1 1 1 0 1 0

0 0 1 0 1 1

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011

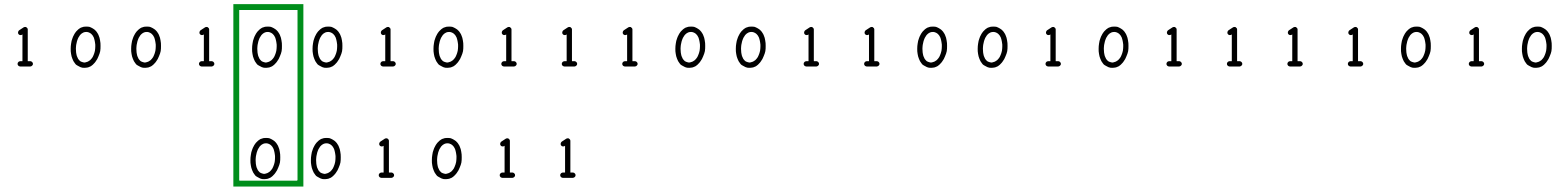
1 0 0 1 0 0 1 0 1 1 1 0 0 1 1 0 0 1 0 1 1 1 1 0 1 0

0 0 1 0 1 1

Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

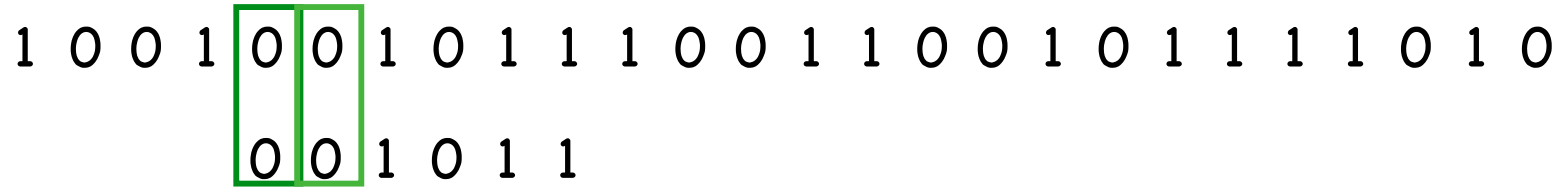
t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

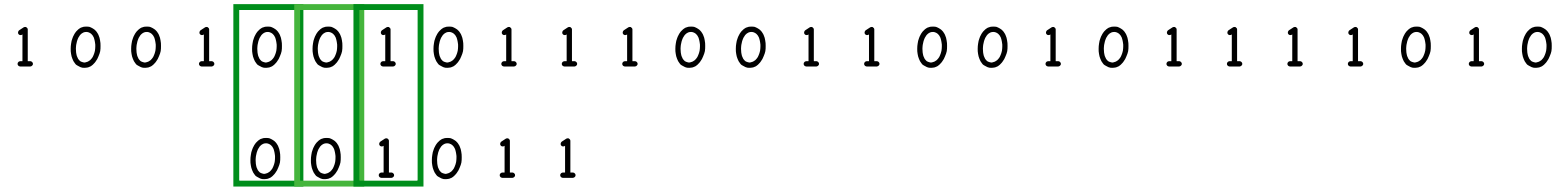
t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

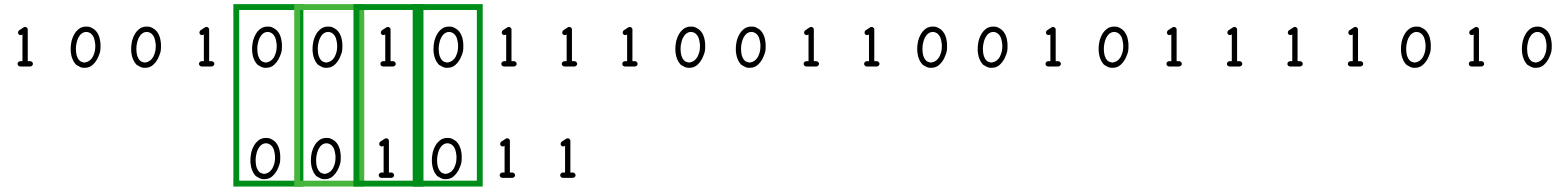
t: 100100101110011001011111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

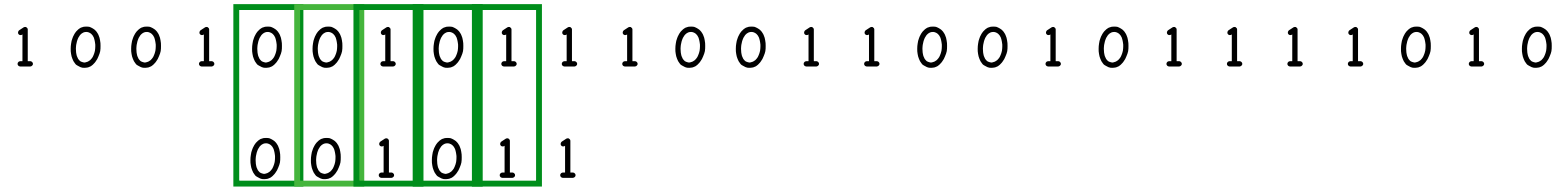
t: 1001001011100110010111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

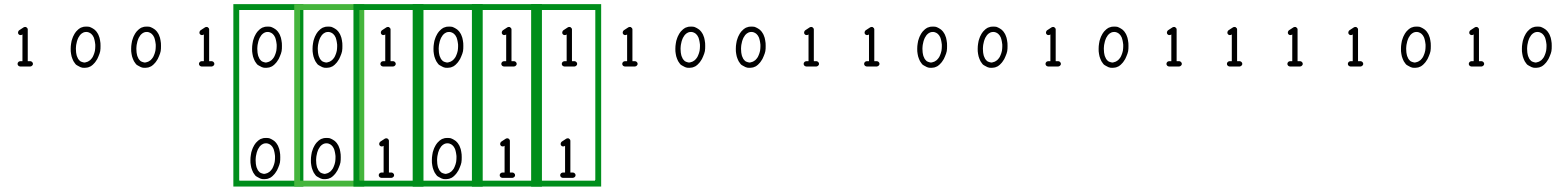
t: 10010010111001100101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

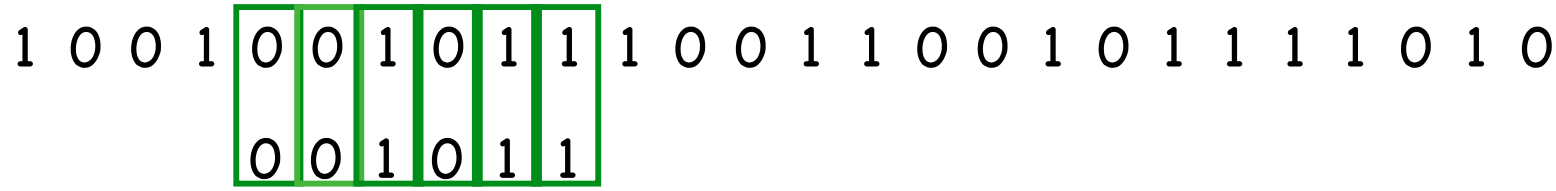
t: 1001001011100110010101111010 p: 001011



Ejemplo:
Matching de Cadenas por Fuerza Bruta

Ejemplo:

t: 10010010111001100101111010 p: 001011





Análisis de String Matching por Fuerza Bruta

```
Algoritmo BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  // Chequea si P es una subcadena de T.
  // Entrada: un arreglo T[0..n-1] de n caracteres, que representa un texto, y un
  // arreglo P[0..m-1], que representa un patrón a buscar en el texto.
  // Salida: el índice del primer caracter en el texto que comienza en una
  // subcadena que matchea con P. -1 si P no es subcadena de T.
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

Análisis de String Matching por Fuerza Bruta

```
Algoritmo BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  // Chequea si P es una subcadena de T.
  // Entrada: un arreglo T[0..n-1] de n caracteres, que representa un texto, y un
  // arreglo P[0..m-1], que representa un patrón a buscar en el texto.
  // Salida: el índice del primer caracter en el texto que comienza en una
  // subcadena que matchea con P. -1 si P no es subcadena de T.
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

¿Porqué hasta n-m?

Análisis de String Matching por Fuerza Bruta

```
Algoritmo BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  // Chequea si P es una subcadena de T.
  // Entrada: un arreglo T[0..n-1] de n caracteres, que representa un texto, y un
  // arreglo P[0..m-1], que representa un patrón a buscar en el texto.
  // Salida: el índice del primer caracter en el texto que comienza en una
  // subcadena que matchea con P. -1 si P no es subcadena de T.
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

¿Porqué hasta n-m?

- ¿Cuál es la eficiencia en tiempo?
- ¿Peor Caso? ¿Mejor Caso?

Análisis de String Matching por Fuerza Bruta

```
Algoritmo BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

Operación de interés: comparación

¿Worst case input?

Análisis de String Matching por Fuerza Bruta

```
Algoritmo BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

Operación de interés: comparación

¿Worst case input?

m comparaciones antes de shiftear, n-m +1 shifteos (no hay matching)

Análisis de String Matching por Fuerza Bruta

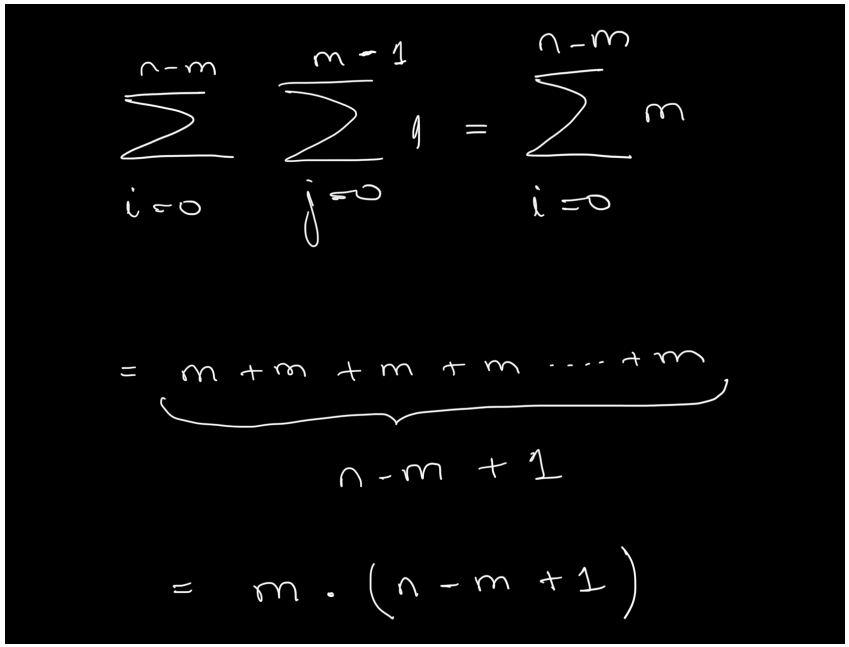
Algoritmo BruteForceStringMatch(char T[0..n-1], char P[0..m-1])

```
for i ← 0 to n-m do
  j ← 0
  while j < m and P[j] = T[i+j] do
    j ← j+1
  if j=m then return i
endfor
return -1
```

Operación de interés: comparación

¿Worst case input?

m comparaciones antes de shiftear, n-m +1 shifteos (no hay matching)



Análisis de String Matching por Fuerza Bruta

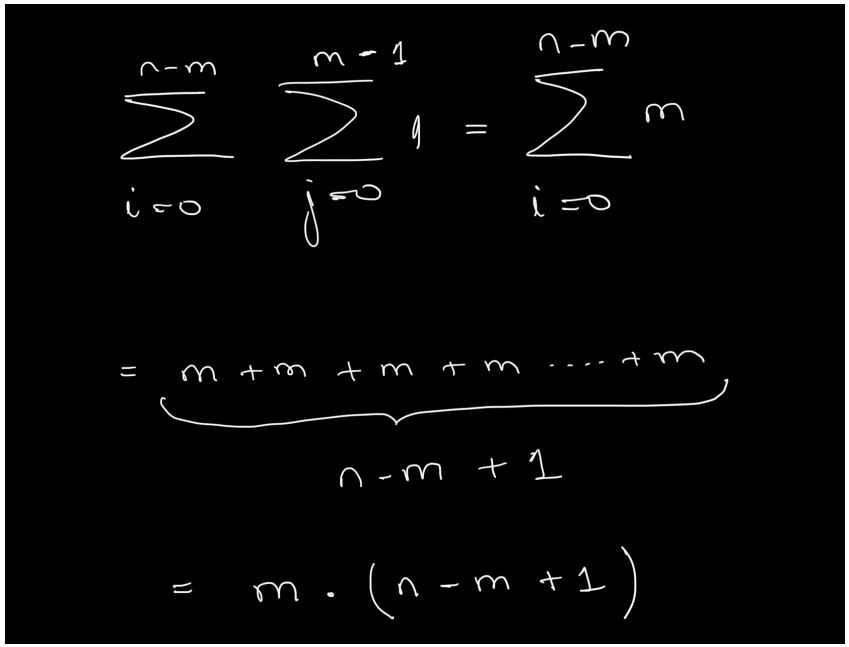
Algoritmo BruteForceStringMatch(char T[0..n-1], char P[0..m-1])

```
for i ← 0 to n-m do
  j ← 0
  while j < m and P[j] = T[i+j] do
    j ← j+1
  if j=m then return i
endfor
return -1
```

Operación de interés: comparación

¿Worst case input?

m comparaciones antes de shiftear, n-m +1 shifteos (no hay matching)



$$O(m \,.\, n)$$

Análisis de String Matching por Fuerza Bruta

```
Algoritmo  BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

Análisis de String Matching por Fuerza Bruta

```
Algoritmo  BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

Operación de interés: comparación

Análisis de String Matching por Fuerza Bruta

```
Algoritmo  BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

Operación de interés: comparación

¿Y el Best Case input?

Análisis de String Matching por Fuerza Bruta

```
Algoritmo BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

Operación de interés: comparación

¿Y el Best Case input?

m comparaciones nunca shiftea

Análisis de String Matching por Fuerza Bruta

```
Algoritmo BruteForceStringMatch(char T[0..n-1], char P[0..m-1])
  for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
      j ← j+1
    if j=m then return i
  endfor
  return -1
```

Operación de interés: comparación

¿Y el Best Case input?

m comparaciones nunca shiftea

$$O(m)$$

Ejemplo 3:
Encontrar el Par de Puntos Más Cercanos

Problema: Encontrar los dos puntos más cercanos en un conjunto de n puntos (en el plano cartesiano).

Ejemplo 3:
Encontrar el Par de Puntos Más Cercanos

Problema: Encontrar los dos puntos más cercanos en un conjunto de n puntos (en el plano cartesiano).

Solución por fuerza bruta:

Computar la distancia entre cada par de puntos (diferentes) del conjunto y retornar los índices de aquellos puntos cuya distancia sea la menor.

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

Algoritmo BruteForceClosestPoints(P)
// Entrada: una lista (representación de conjuntos) P de n (n>=2) puntos (x_i,y_i).
// Salida: Los índices index1 e index2 del par de puntos de P más cercanos
// entre sí.
dmin ← +infinito
for i ← 1 **to** n-1 **do**
  for j ← i+1 **to** n **do**
    d ← sqrt((x_i-x_j)² + (y_i-y_j)²)
    if d < dmin **then** dmin ← d; index1 ← i; index2 ← j
return index1, index2

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

¿Cuál es la eficiencia en tiempo?

Algoritmo BruteForceClosestPoints(P)
  dmin ← +infinito
  for i ← 1 **to** n-1 **do**
    for j ← i+1 **to** n **do**
      d ← sqrt($(x_i-x_j)^2 + (y_i-y_j)^2$)
      if d < dmin **then** dmin ← d; index1 ← i; index2 ← j
  return index1, index2

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

¿Cuál es la eficiencia en tiempo?

Algoritmo BruteForceClosestPoints(P)
  dmin ← +infinito
  for i ← 1 **to** n-1 **do**
    for j ← i+1 **to** n **do**
      d ← sqrt($(x_i-x_j)^2 + (y_i-y_j)^2$)
      if d < dmin **then** dmin ← d; index1 ← i; index2 ← j
  return index1, index2

Operación de interés: cálculo de la raíz

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

¿Cuál es la eficiencia en tiempo?

Algoritmo BruteForceClosestPoints(P)
  dmin ← +infinito
  for i ← 1 **to** n-1 **do**
    for j ← i+1 **to** n **do**
      d ← sqrt($(x_i-x_j)^2 + (y_i-y_j)^2$)
      if d < dmin **then** dmin ← d; index1 ← i; index2 ← j
  return index1, index2

Operación de interés: cálculo de la raíz

Cantidad de operaciones dependen solo del tamaño de la entrada: número de pares

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

```
Algoritmo BruteForceClosestPoints(P)
    dmin ← +infinito
    for i ← 1 to n-1 do
        for j ← i+1 to n do
            d ← sqrt((xi-xj)² + (yi-yj)²)
            if d < dmin then dmin ← d; index1 ← i; index2 ← j
    return index1, index2
```

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

```
Algoritmo BruteForceClosestPoints(P)
    dmin ← +infinito
    for i ← 1 to n-1 do
        for j ← i+1 to n do
            d ← sqrt((xi-xj)² + (yi-yj)²)
            if d < dmin then dmin ← d; index1 ← i; index2 ← j
    return index1, index2
```

$$O(n^2)$$

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

```
Algoritmo BruteForceClosestPoints(P)
  dmin ← +infinito
  for i ← 1 to n-1 do
    for j ← i+1 to n do
      d ← sqrt((xi-xj)² + (yi-yj)²)
      if d < dmin then dmin ← d; index1 ← i; index2 ← j
  return index1, index2
```

$O(n^2)$

¿Cómo podríamos mejorarlo?

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

```
Algoritmo BruteForceClosestPoints(P)
  dmin ← +infinito
  for i ← 1 to n-1 do
    for j ← i+1 to n do
      d ← sqrt((xi-xj)² + (yi-yj)²)
      if d < dmin then dmin ← d; index1 ← i; index2 ← j
  return index1, index2
```

$O(n^2)$

¿Cómo podríamos mejorarlo?

Podríamos evitar el cálculo de la raíz!

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

```
Algoritmo BruteForceClosestPoints(P)
  dmin ← +infinito
  for i ← 1 to n-1 do
    for j ← i+1 to n do
      d ← sqrt((xi-xj)² + (yi-yj)²)
      if d < dmin then dmin ← d; index1 ← i; index2 ← j
  return index1, index2
```

$O(n^2)$

¿Cómo podríamos mejorarlo?

Podríamos evitar el cálculo de la raíz!

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1$$

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1} n - (i+1) + 1 =$$

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1} n - (i+1) + 1 = \sum_{i=1}^{n-1} n - i - 1 + 1 = \sum_{i=1}^{n-1} n - i =$$

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1} n - (i+1) + 1 = \sum_{i=1}^{n-1} n - i - 1 + 1 = \sum_{i=1}^{n-1} n - i =$$

$$\sum_{i=1}^{n-1} i =$$

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1} n - (i+1) + 1 = \sum_{i=1}^{n-1} n - i - 1 + 1 = \sum_{i=1}^{n-1} n - i =$$

$$\sum_{i=1}^{n-1} i = \frac{(n-1) \, . \, n}{2}$$

Análisis de Cálculo de Pares Más Cercanos por Fuerza Bruta

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1} n - (i+1) + 1 = \sum_{i=1}^{n-1} n - i - 1 + 1 = \sum_{i=1}^{n-1} n - i =$$

$$\sum_{i=1}^{n-1} i = \frac{(n-1) \, . \, n}{2}$$

$$O(n^2)$$

Pros y Contras de la Fuerza Bruta

Pros

- amplia aplicabilidad
- simplicidad
- provee algoritmos razonables para una clase importante de problemas (e.g., multiplicación de matrices, ordenamiento, búsqueda, string matching)

Contras

- rara vez da como resultado algoritmos eficientes
- algunos algoritmos de ordenamiento son inaceptablemente lentos
- no tan "constructiva" como otras técnicas de diseño

Búsqueda Exhaustiva

Una solución por fuerza bruta a un problema que involucra una búsqueda de un elemento con alguna propiedad particular, usualmente entre una cantidad combinatoria de objetos (e.g., permutaciones, combinaciones, subconjuntos de un conjunto, etc).

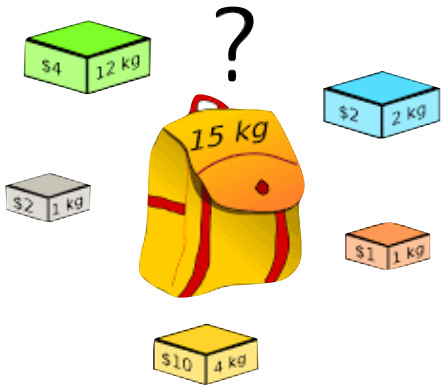
Búsqueda Exhaustiva

Una solución por fuerza bruta a un problema que involucra una búsqueda de un elemento con alguna propiedad particular, usualmente entre una cantidad combinatoria de objetos (e.g., permutaciones, combinaciones, subconjuntos de un conjunto, etc).

Método:

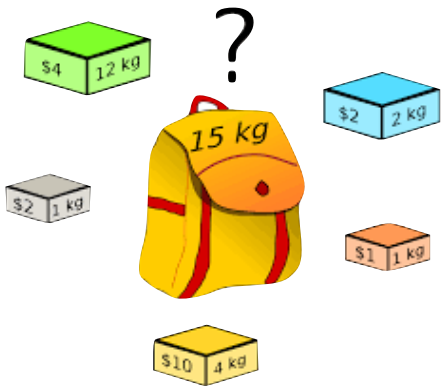
- generar una lista de todas las posibles soluciones al problema de una manera sistemática
- evaluar las potenciales soluciones una por una, desechando las que no sirvan; en el caso de problemas de optimización, se debe llevar cuenta del mejor objeto encontrado hasta el momento
- cuando se termine la búsqueda, retornar la solución encontrada

El problema de la Mochila



El problema de la Mochila

El problema de la mochila es un problema clásico de optimización. Consideremos el caso en el que contamos con n items, cada uno con peso w_i y valor v_i. Dada una mochila de capacidad (peso máximo soportado) W, **encontrar el subconjunto de items que quepa en la mochila, y que maximice la suma de los valores del subconjunto.**



El Problema de la Mochila

El Problema de la Mochila

Ejemplo: Supongamos que tenemos una mochila de capacidad W=16, y los siguientes items:

El Problema de la Mochila

Ejemplo: Supongamos que tenemos una mochila de capacidad W=16, y los siguientes items:

item	peso	valor
1	2	$20
2	5	$30
3	10	$50
4	5	$10

El Problema de la Mochila por Fuerza Bruta

Para resolverlo por fuerza bruta, consideremos todos los posibles subconjuntos de items:

El Problema de la Mochila por Fuerza Bruta

Para resolverlo por fuerza bruta, consideremos todos los posibles subconjuntos de items:

Subconjunto	Peso total	Valor total

El Problema de la Mochila por Fuerza Bruta

Para resolverlo por fuerza bruta, consideremos todos los posibles subconjuntos de items:

Subconjunto	Peso total	Valor total
{1}	2	$20
{2}	5	$30
{3}	10	$50
{4}	5	$10
{1,2}	7	$50
{1,3}	12	$70
{1,4}	7	$30
{2,3}	15	$80
{2,4}	10	$40
{3,4}	15	$60
{1,2,3}	17	NO REALIZABLE
{1,2,4}	12	$60
{1,3,4}	17	NO REALIZABLE
{2,3,4}	20	NO REALIZABLE
{1,2,3,4}	22	NO REALIZABLE

El Problema de la Mochila por Fuerza Bruta

Para resolverlo por fuerza bruta, consideremos todos los posibles subconjuntos de items:

item	peso	valor
1	2	$20
2	5	$30
3	10	$50
4	5	$10

Subconjunto	Peso total	Valor total
{1}	2	$20
{2}	5	$30
{3}	10	$50
{4}	5	$10
{1,2}	7	$50
{1,3}	12	$70
{1,4}	7	$30
{2,3}	15	$80
{2,4}	10	$40
{3,4}	15	$60
{1,2,3}	17	NO REALIZABLE
{1,2,4}	12	$60
{1,3,4}	17	NO REALIZABLE
{2,3,4}	20	NO REALIZABLE
{1,2,3,4}	22	NO REALIZABLE

SUBSETS

Dado un conjunto calcula todos los subconjuntos que puedo formar con él - Solución Iterativa (bottom up)

```
Function subSets(conjunto) {
    powerSet = { {} } // Empezamos con el conjunto vacío adentro
    for (valor : conjunto) {
        new powerSetAux = { } // Conjunto vacío temporal
        for (t : powerSet) {
            new nuevoSubSet = t u {valor}
            powerSetAux = powerSetAux u {nuevoSubSet}
        }

        powerSet = powerSet u powerSetAux
   }

   return powerSet
}
```

Análisis de Tiempo de Ejecución

Siendo n la cantidad de elementos de conjunto

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=1}^{2^i} n$$

asumo lineal si los conj están implementadas eficientemente.

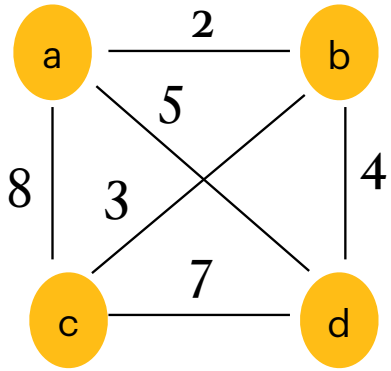
$$T(n) = \sum_{i=0}^{n-1} n \cdot 2^i = n \cdot \sum_{i=0}^{n-1} 2^i$$

$$n \cdot \frac{2^{n-1+1} - 1}{2 - 1} = n \cdot (2^n - 1) = \Theta(2^n)$$

Geometric series:

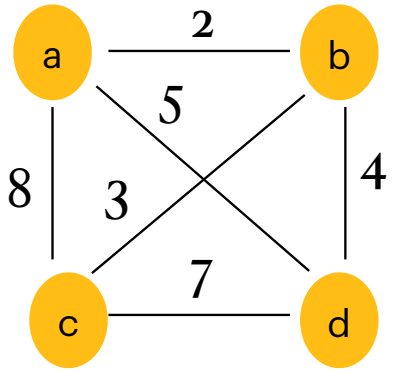
$$\sum_{i=0}^{n} c^i = \frac{c^{n+1} - 1}{c - 1}, \quad c \neq 1,$$

El Problema del Agente Viajero



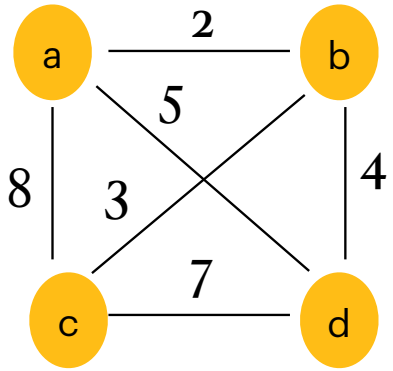
El Problema del Agente Viajero

Dadas n ciudades con distancias conocidas entre cada par, encontrar el tour más corto que pase por todas las ciudades exactamente una vez, y que vuelva al lugar de partida.



El Problema del Agente Viajero

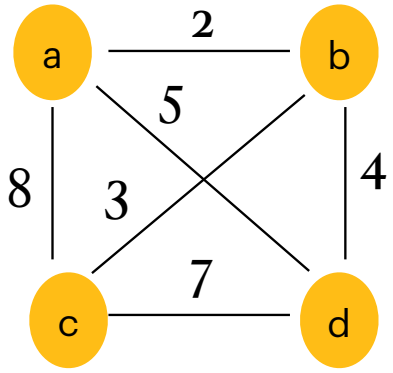
Dadas n ciudades con distancias conocidas entre cada par, encontrar el tour más corto que pase por todas las ciudades exactamente una vez, y que vuelva al lugar de partida.



El Problema del Agente Viajero

Dadas n ciudades con distancias conocidas entre cada par, encontrar el tour más corto que pase por todas las ciudades exactamente una vez, y que vuelva al lugar de partida.

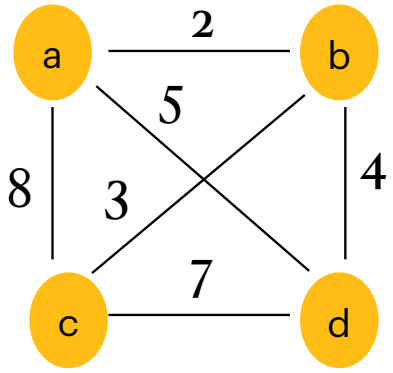
Si se observa que los datos de entrada pueden formalizarse como un grafo (no dirigido) conexo con pesos en los arcos, el problema equivale a encontrar el circuito hamiltoniano mínimo en el grafo.



El Problema del Agente Viajero

Dadas n ciudades con distancias conocidas entre cada par, encontrar el tour más corto que pase por todas las ciudades exactamente una vez, y que vuelva al lugar de partida.

Si se observa que los datos de entrada pueden formalizarse como un grafo (no dirigido) conexo con pesos en los arcos, el problema equivale a encontrar el circuito hamiltoniano mínimo en el grafo.

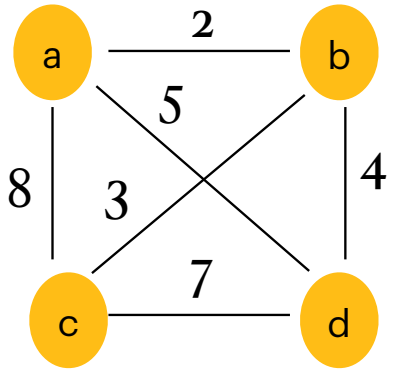


El Problema del Agente Viajero

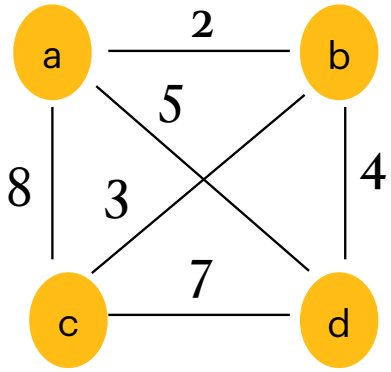
Dadas n ciudades con distancias conocidas entre cada par, encontrar el tour más corto que pase por todas las ciudades exactamente una vez, y que vuelva al lugar de partida.

Si se observa que los datos de entrada pueden formalizarse como un grafo (no dirigido) conexo con pesos en los arcos, el problema equivale a encontrar el circuito hamiltoniano mínimo en el grafo.

Ejemplo:

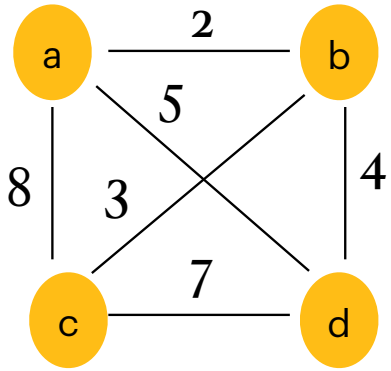


El Problema del Agente Viajero



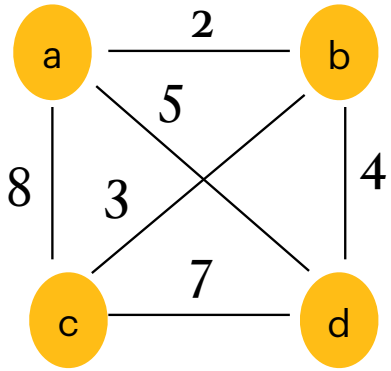
El Problema del Agente Viajero

Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



El Problema del Agente Viajero

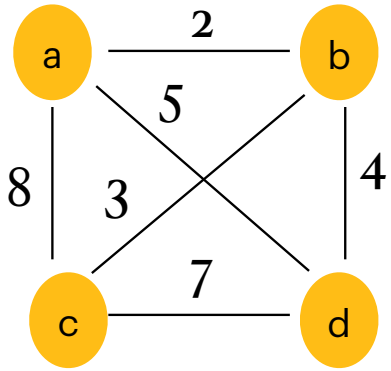
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo

El Problema del Agente Viajero

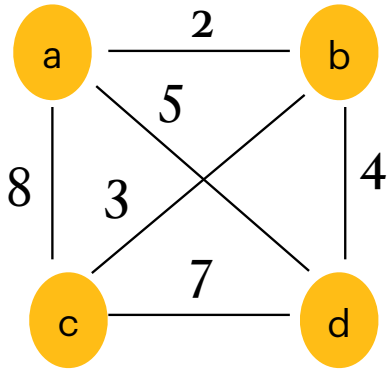
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17

El Problema del Agente Viajero

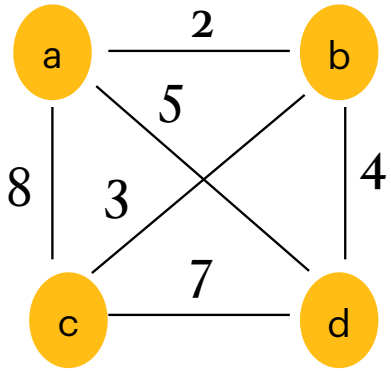
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21

El Problema del Agente Viajero

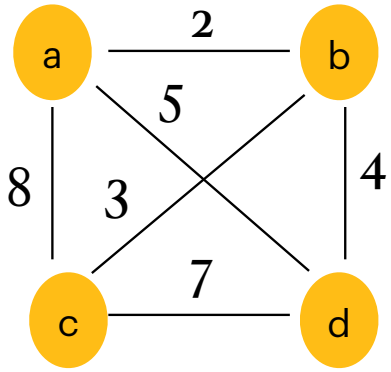
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20

El Problema del Agente Viajero

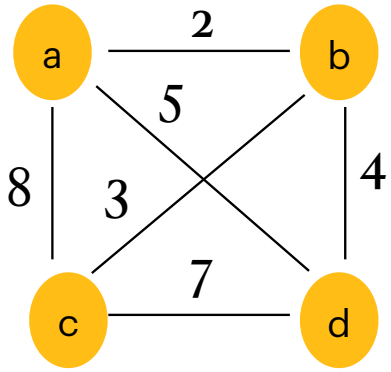
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21

El Problema del Agente Viajero

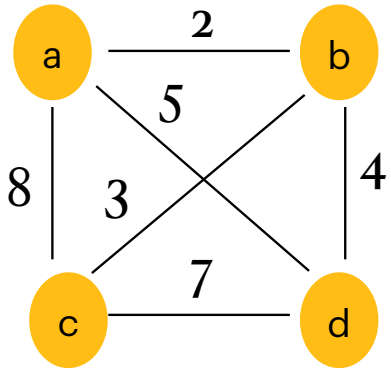
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20

El Problema del Agente Viajero

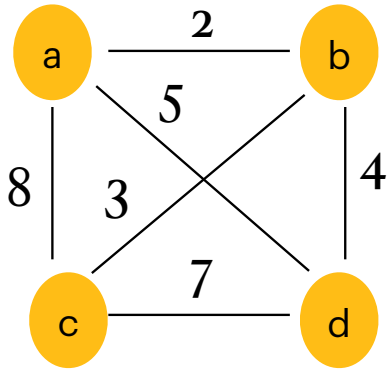
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17

El Problema del Agente Viajero

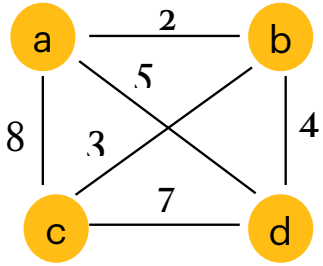
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17

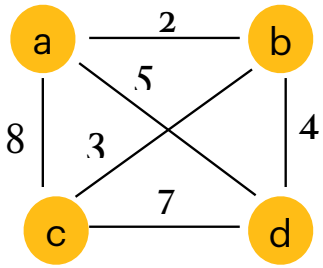
Las soluciones son los circuitos con costo 17.

El Problema del Agente Viajero



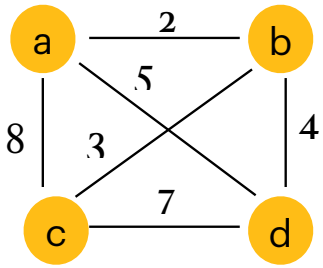
El Problema del Agente Viajero

Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



El Problema del Agente Viajero

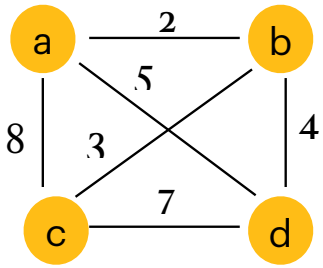
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo

El Problema del Agente Viajero

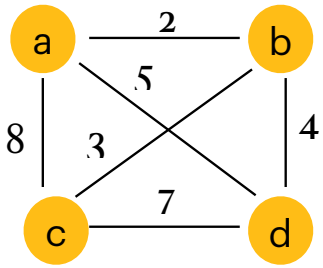
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17

El Problema del Agente Viajero

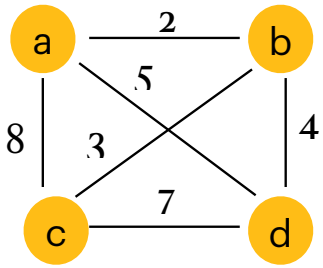
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21

El Problema del Agente Viajero

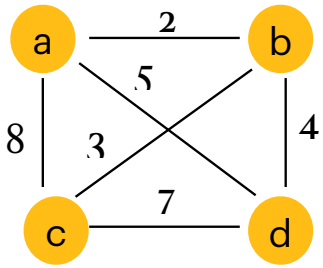
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20

El Problema del Agente Viajero

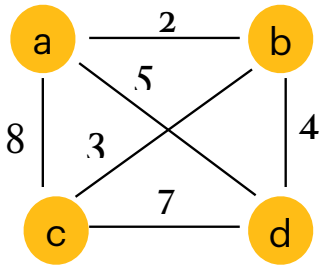
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21

El Problema del Agente Viajero

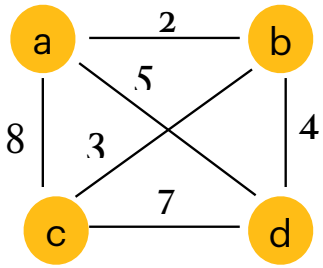
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20

El Problema del Agente Viajero

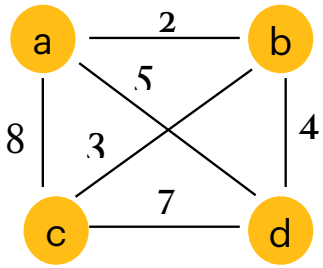
Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17

El Problema del Agente Viajero

Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:

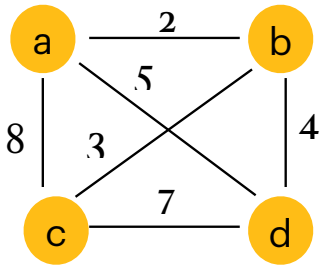


Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17

La cantidad de caminos que debemos mirar corresponde a todas las permutaciones de n-1 nodos intermedios: (n-1)!

El Problema del Agente Viajero

Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



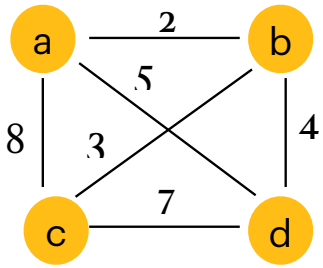
Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17

$\theta(n!)$

La cantidad de caminos que debemos mirar corresponde a todas las permutaciones de n-1 nodos intermedios: (n-1)!

El Problema del Agente Viajero

Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17

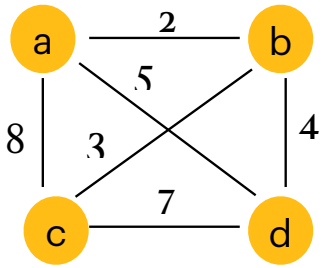
$\theta(n!)$

La cantidad de caminos que debemos mirar corresponde a todas las permutaciones de n-1 nodos intermedios: (n-1)!

Hay caminos que difieren en su dirección, por lo que podríamos optimizar y cortar el número de permutaciones a la mitad: $\frac{(n-1)!}{2}$

El Problema del Agente Viajero

Para resolverlo por fuerza bruta, consideremos todos los circuitos posibles, y sus pesos correspondientes:



Tour	Costo
a→b→c→d→a	2+3+7+5 = 17
a→b→d→c→a	2+4+7+8 = 21
a→c→b→d→a	8+3+4+5 = 20
a→c→d→b→a	8+7+4+2 = 21
a→d→b→c→a	5+4+3+8 = 20
a→d→c→b→a	5+7+3+2 = 17

$\theta(n!)$

La cantidad de caminos que debemos mirar corresponde a todas las permutaciones de n-1 nodos intermedios: (n-1)!

Hay caminos que difieren en su dirección, por lo que podríamos optimizar y cortar el número de permutaciones a la mitad: $\frac{(n-1)!}{2}$

Si no limitamos el nodo de salida con a, entonces esa cantidad la tengo que multiplicar por n n.(n-1)!

Problema de Coloreo de Grafos

Problema de Coloreo de Grafos

Dado un grafo no dirigido, colorear sus nodos con la mínima cantidad de colores de tal manera que nunca dos nodos adyacentes queden coloreados con el mismo color.

Una Propuesta para el Problema de Coloreo de Grafos

Una Propuesta para el Problema de Coloreo de Grafos

Una propuesta razonable para resolver el problema de coloreo de grafos con una cantidad mínima de colores es la siguiente:

Una Propuesta para el Problema de Coloreo de Grafos

Una propuesta razonable para resolver el problema de coloreo de grafos con una cantidad mínima de colores es la siguiente:

> Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes. Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos. Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.

Una Propuesta para el Problema de Coloreo de Grafos

Una propuesta razonable para resolver el problema de coloreo de grafos con una cantidad mínima de colores es la siguiente:

> Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes. Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos. Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.

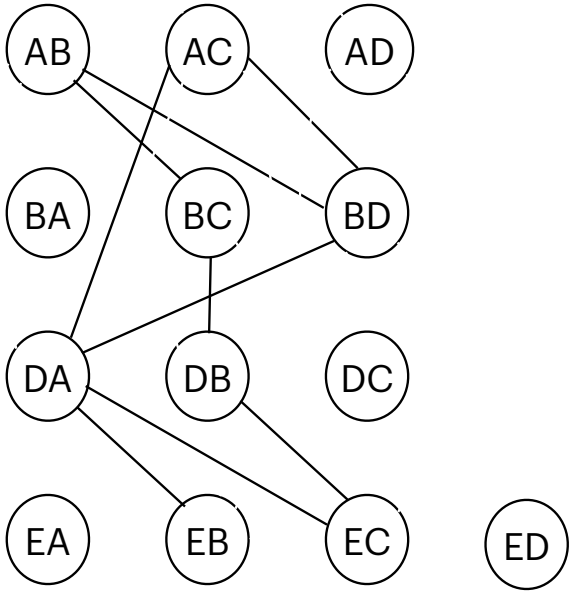
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



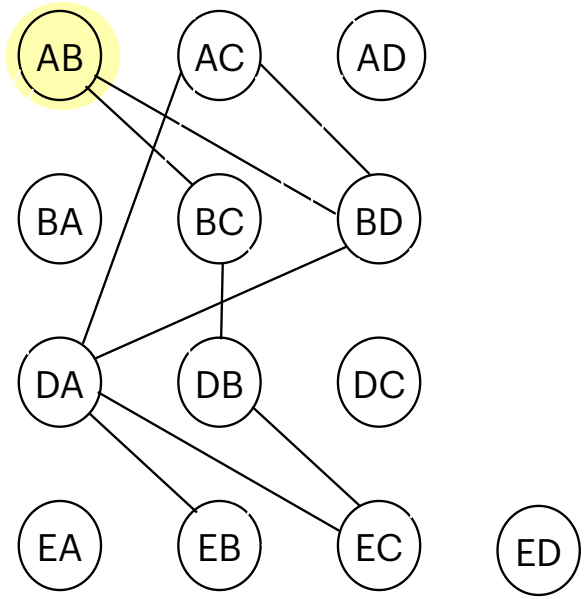
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



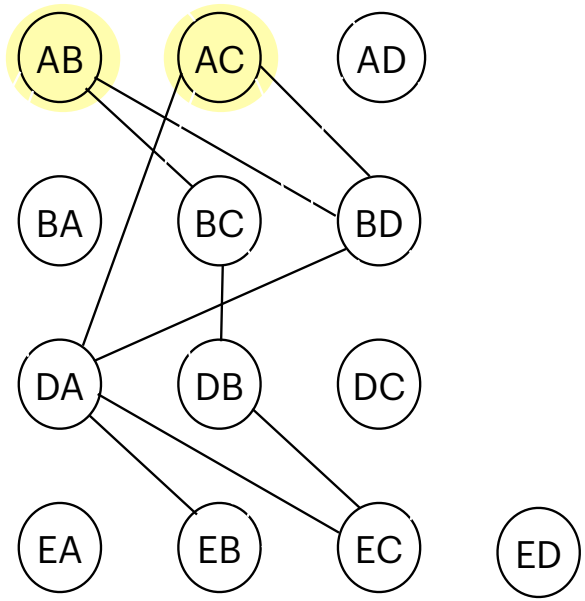
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



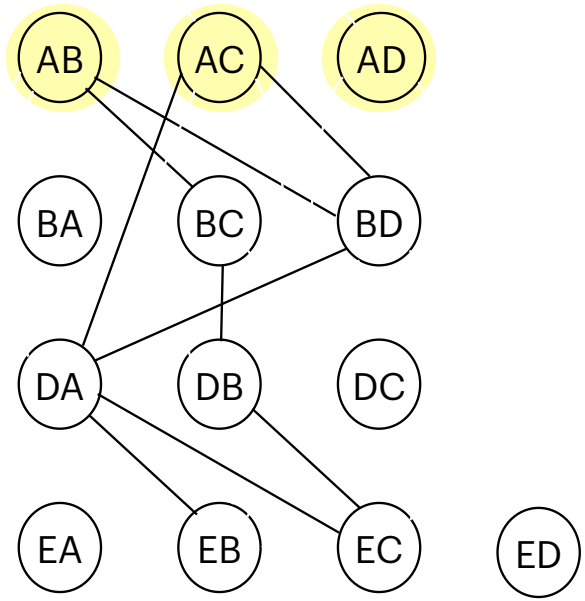
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



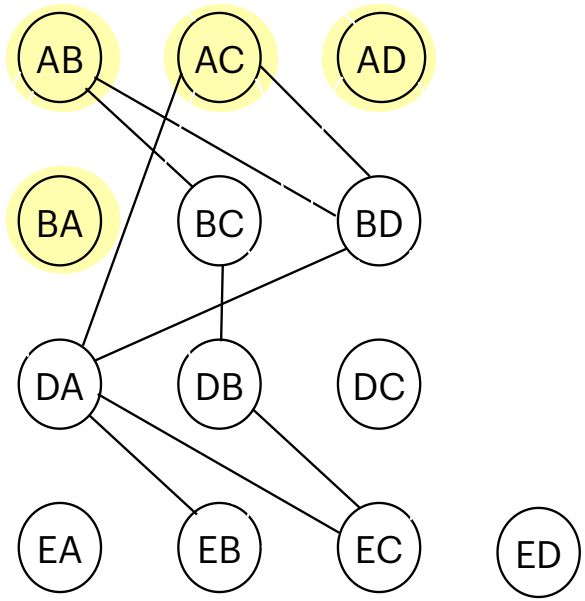
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



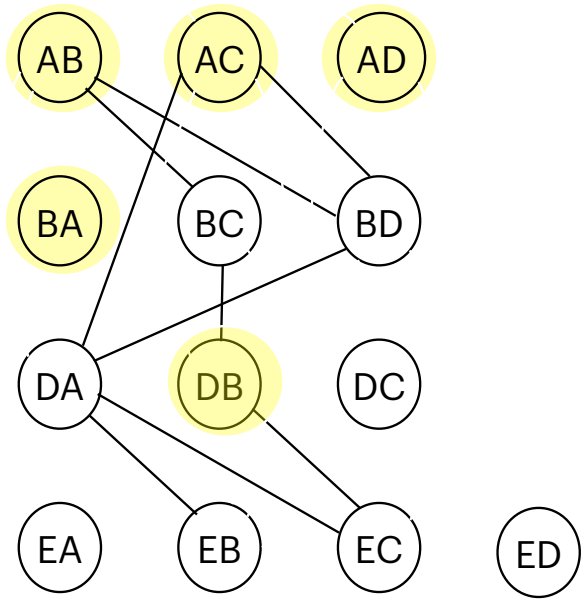
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



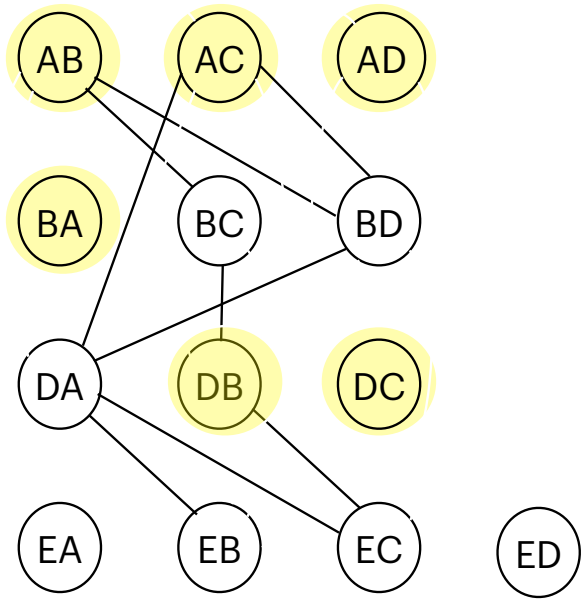
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



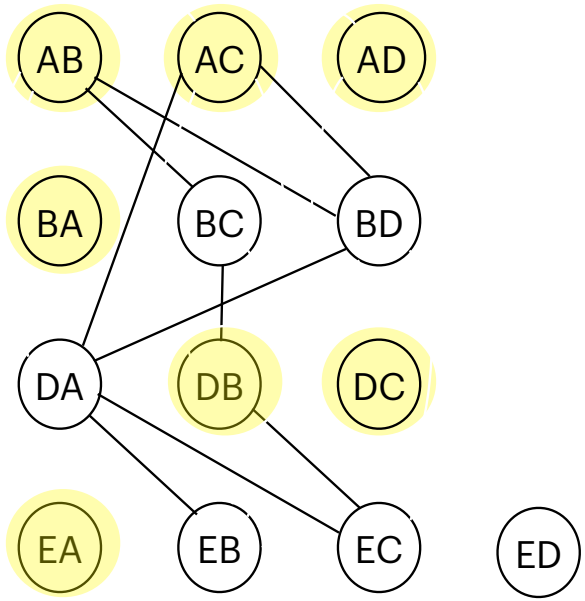
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



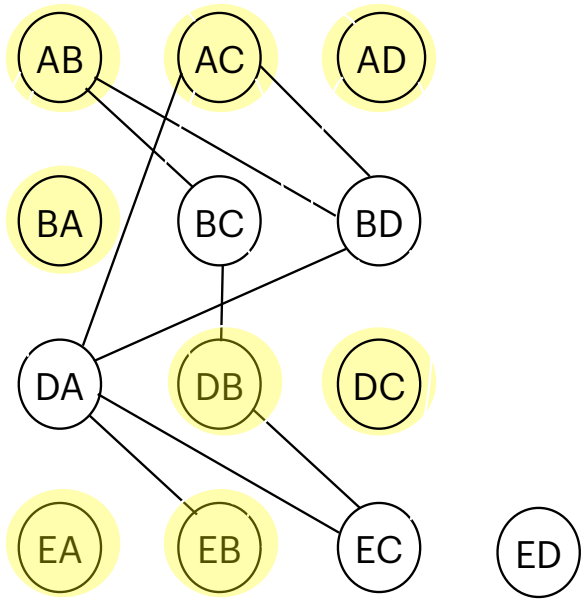
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



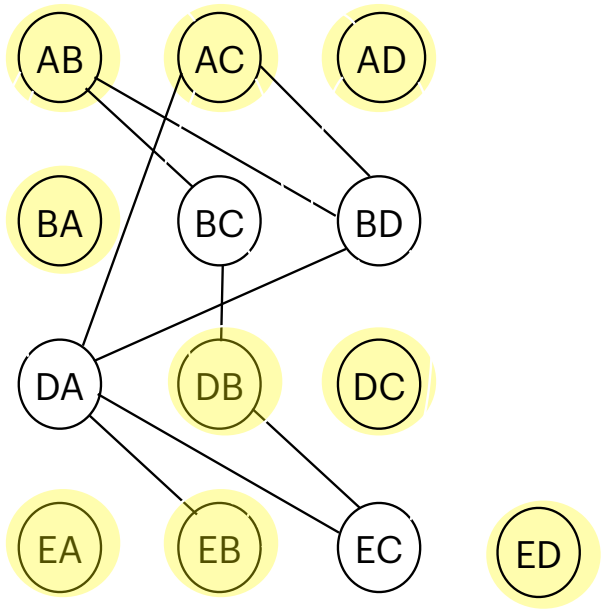
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



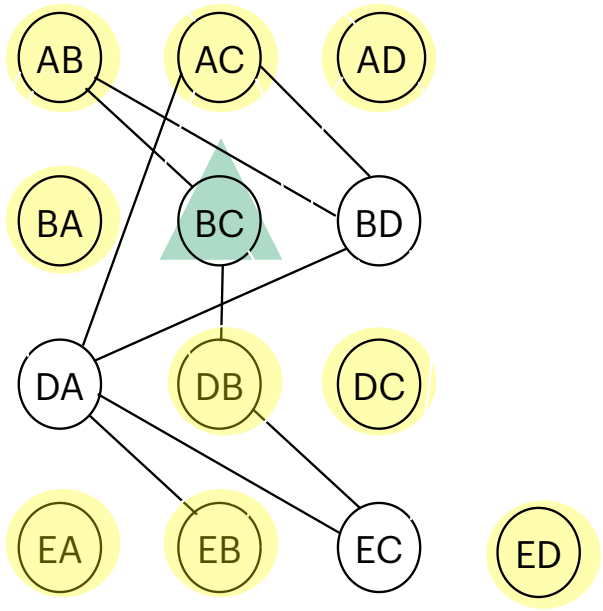
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



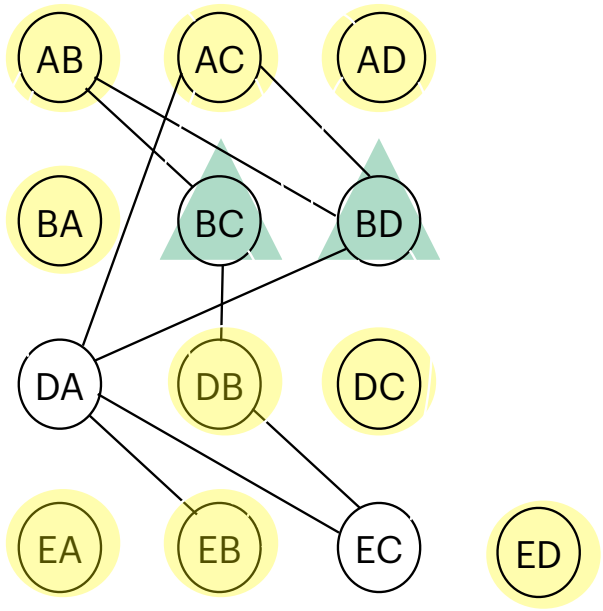
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



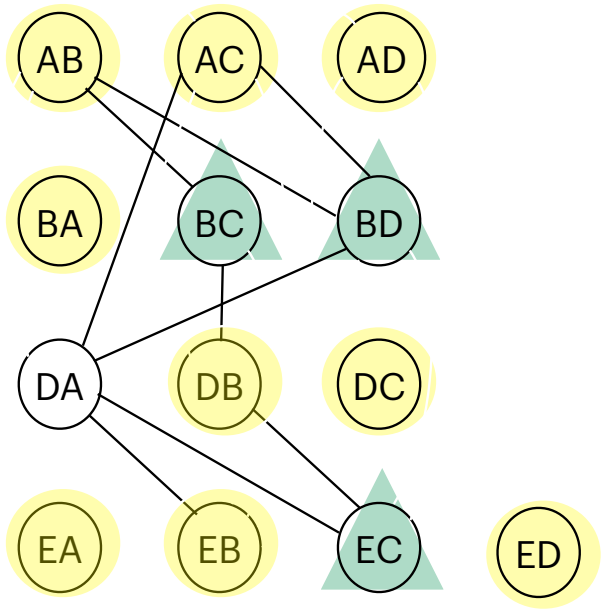
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



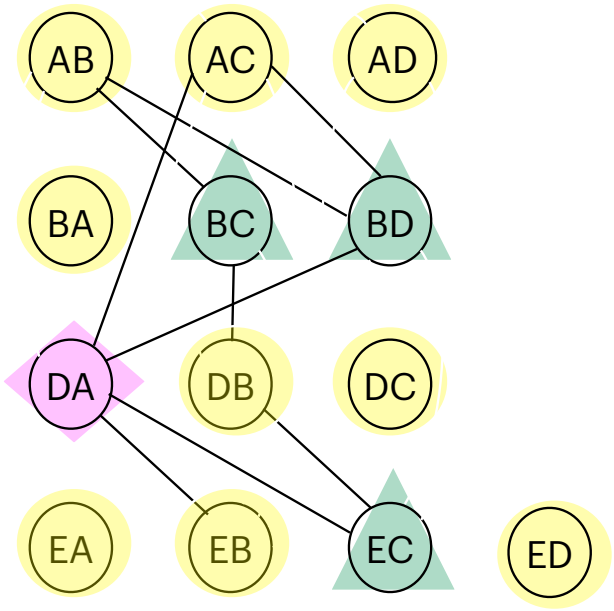
Una Propuesta para el Problema de Coloreo de Grafos

Veamos un ejemplos de cómo funciona la propuesta descripta:

Comenzar a visitar los nodos en orden, coloreando todos los nodos que se pueda con un color, sin causar conflictos entre nodos adyacentes.

Si quedan nodos sin colorear, tomar un nuevo color y repetir el coloreo de tantos nodos no coloreados como se pueda, nuevamente sin causar conflictos.

Continuar el proceso con nuevos colores hasta que ya no queden nodos sin colorear.



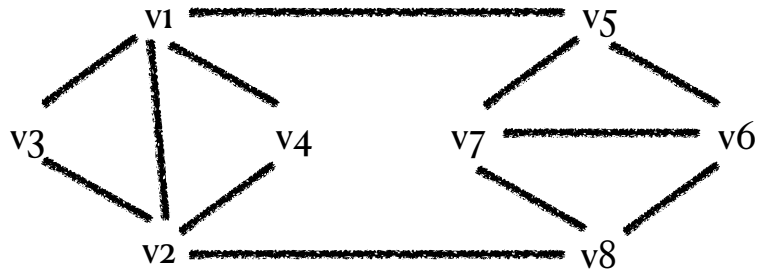
Una Propuesta para el Problema de Coloreo de Grafos

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

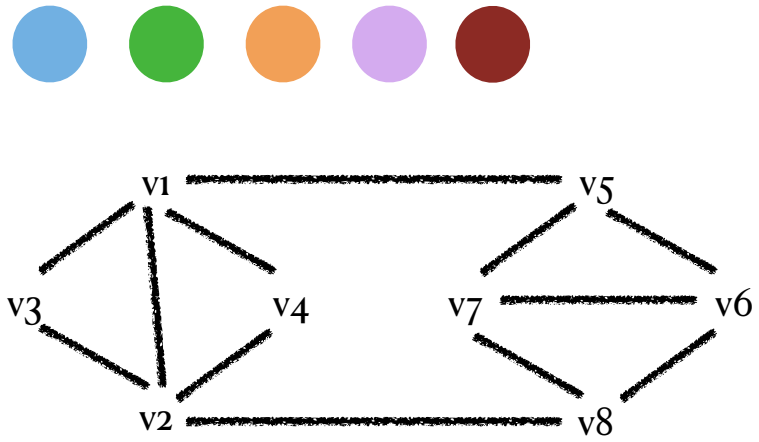
Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.



Una Propuesta para el Problema de Coloreo de Grafos

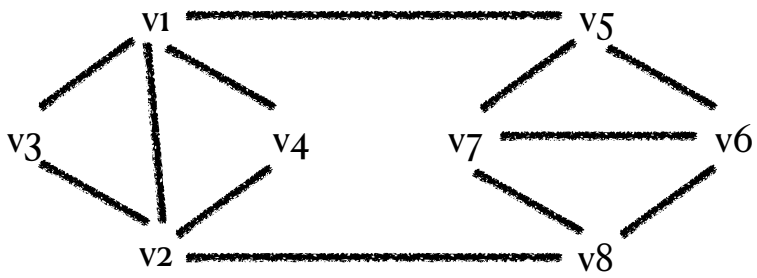
El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.



Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

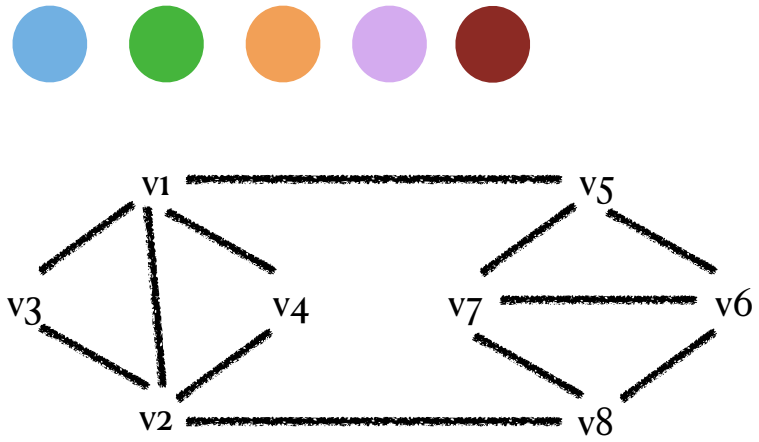




Orden de visita

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

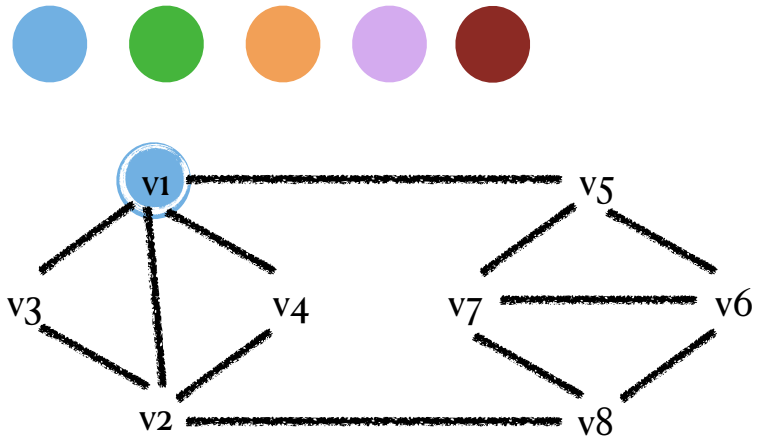


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

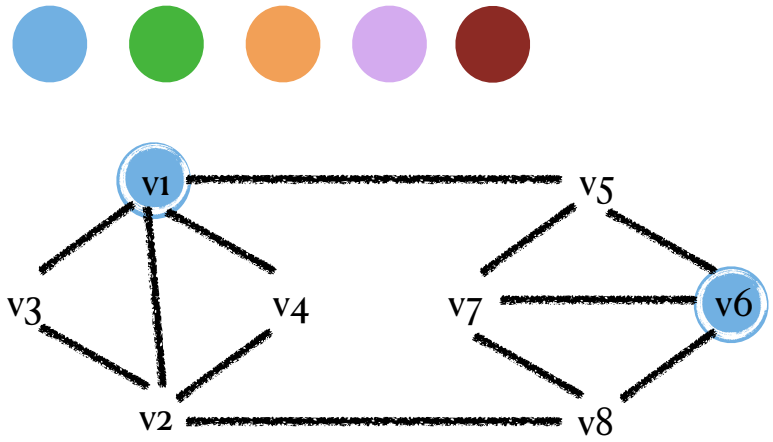


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

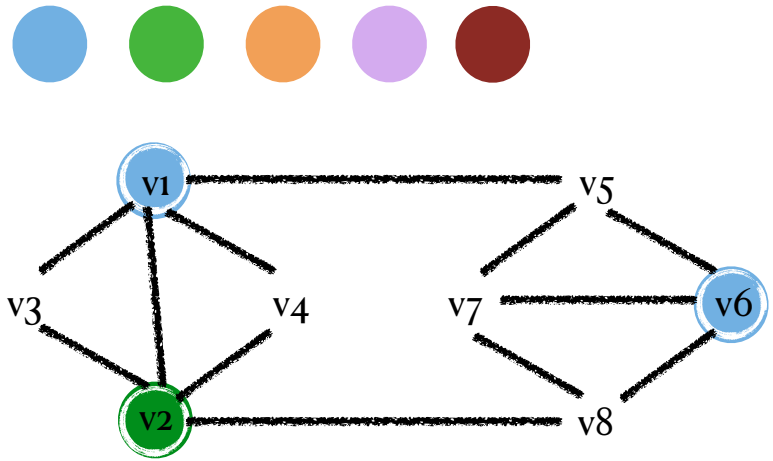


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

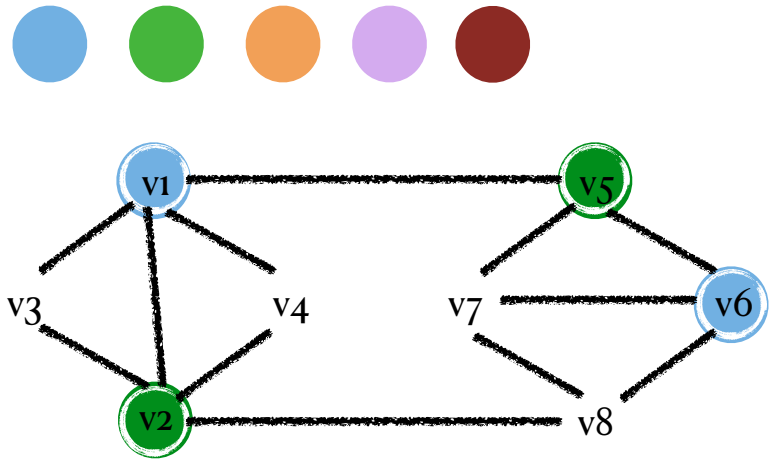


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

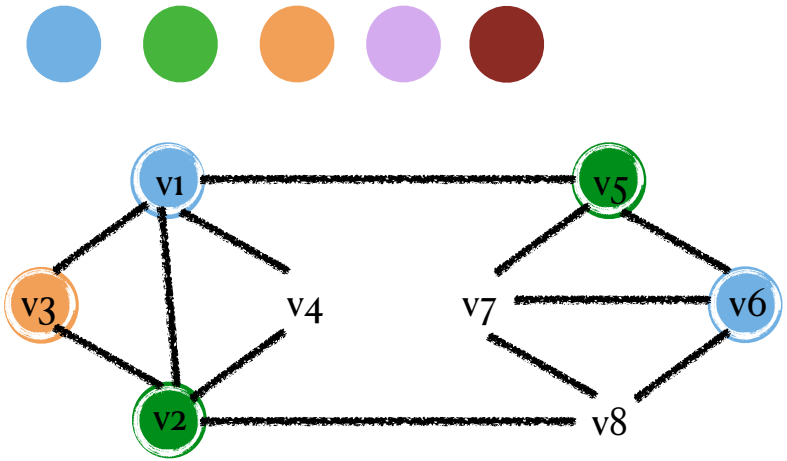


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

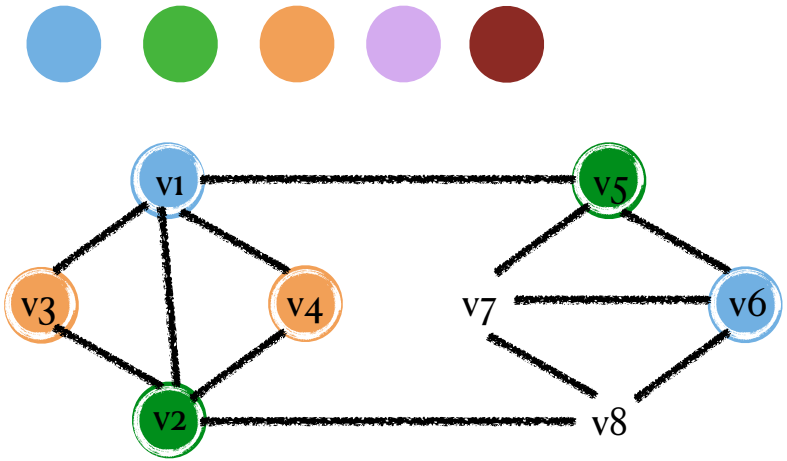


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

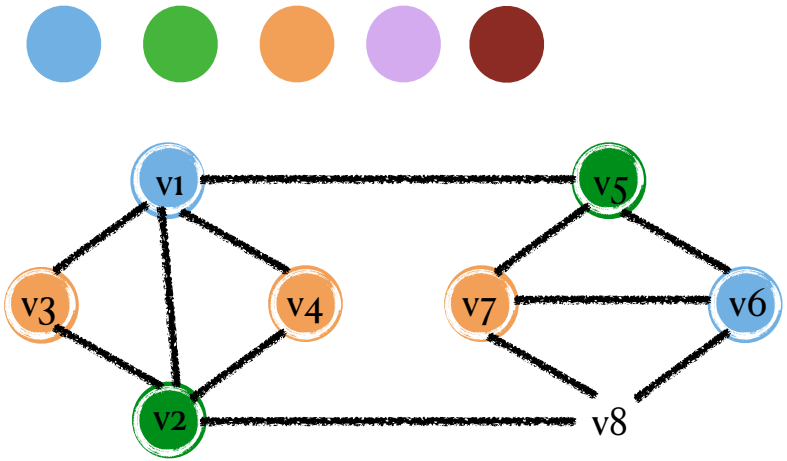


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

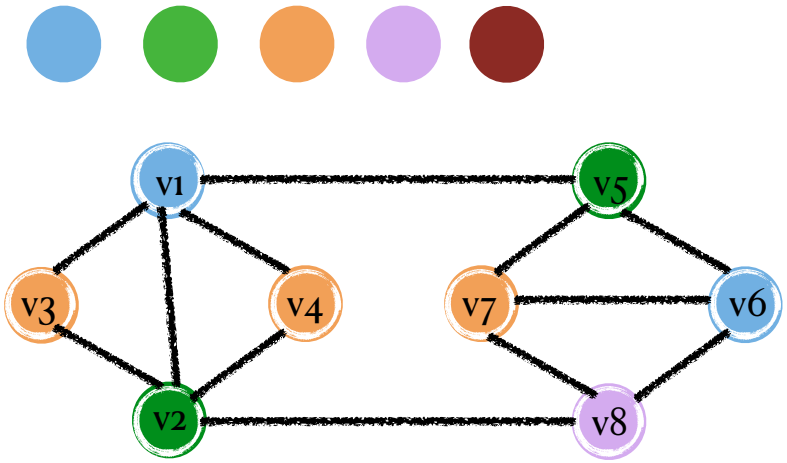


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

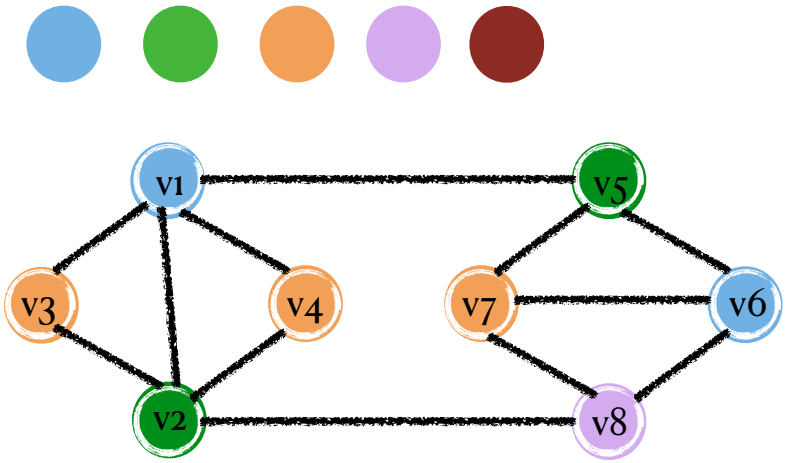


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

Una Propuesta para el Problema de Coloreo de Grafos

El siguiente ejemplo muestra que la solución al problema del coloreo de grafos que propusimos no es óptima.

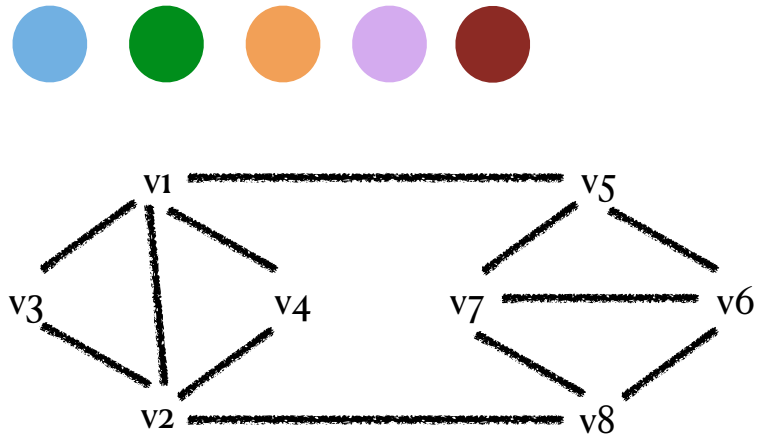


Orden de visita

[v1, v2, v3, v4, v5, v6, v7, v8]

4 colores!

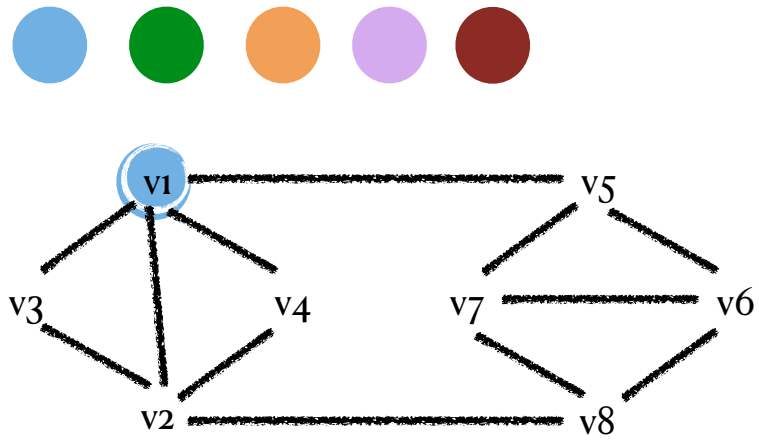
Una Propuesta para el Problema de Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

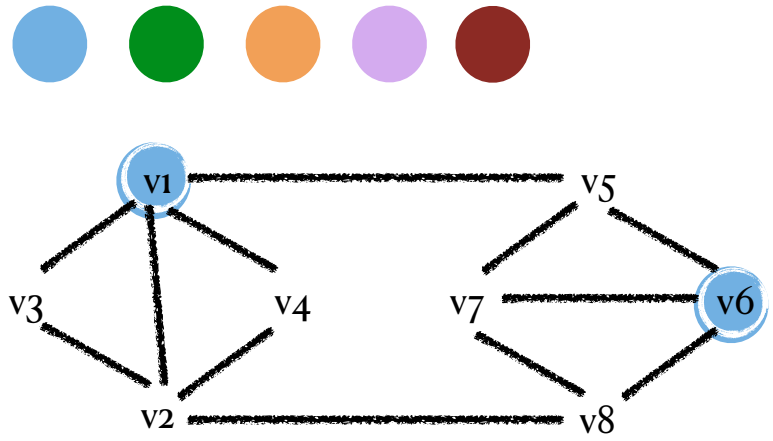
Una Propuesta para el Problema de Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

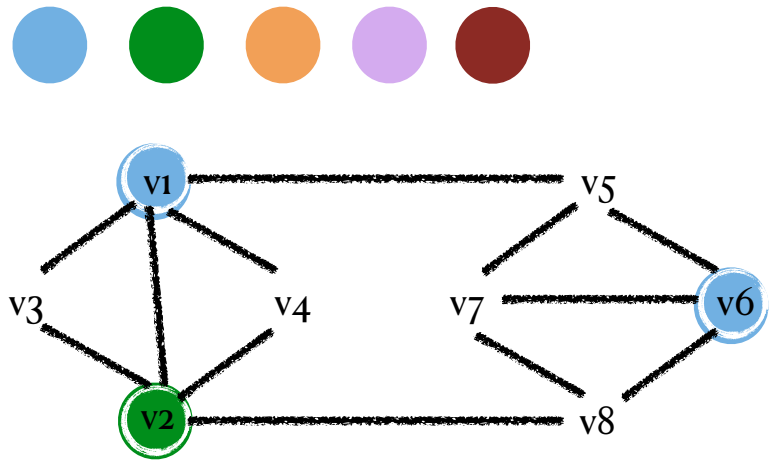
Una Propuesta para el Problema de Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

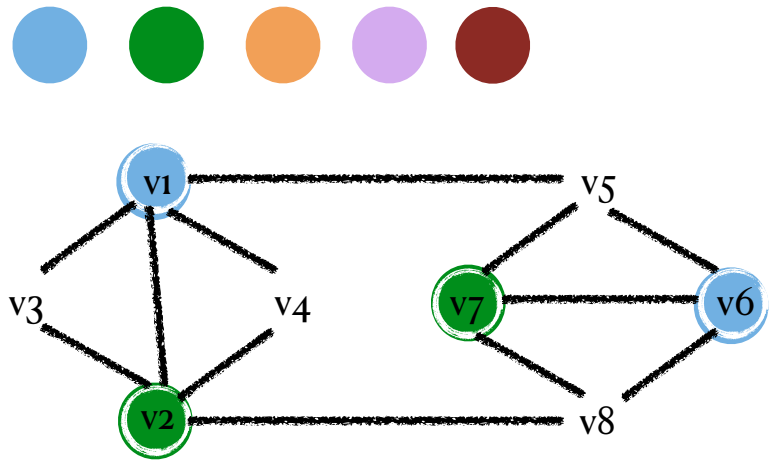
Una Propuesta para el Problema de Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

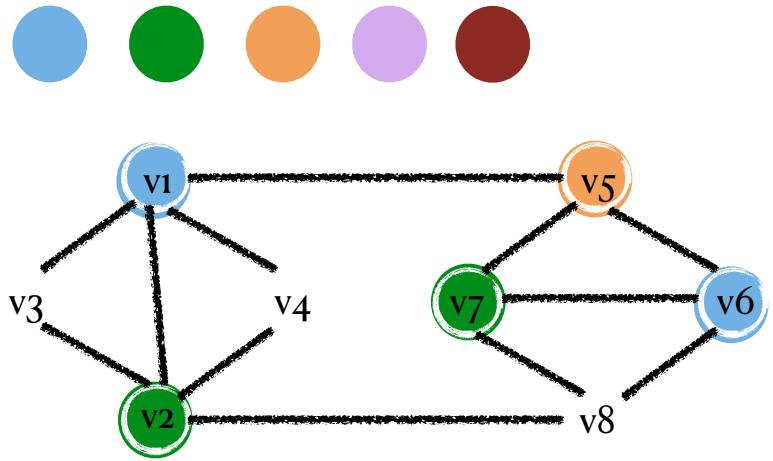
Una Propuesta para el Problema de Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

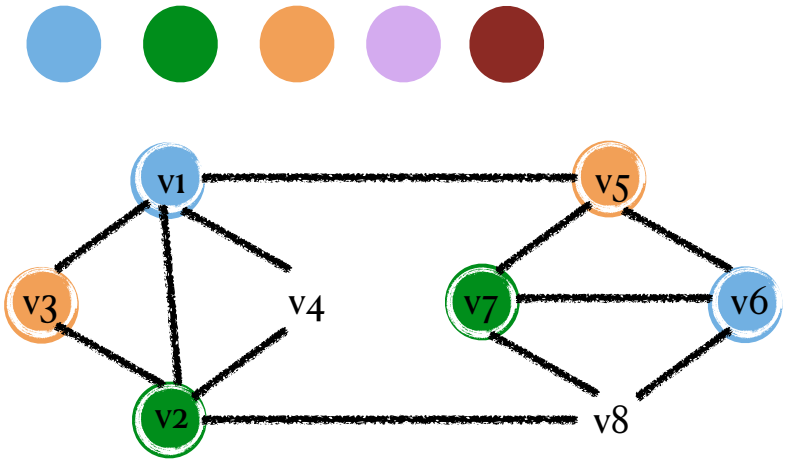
Una Propuesta para el Problema de Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

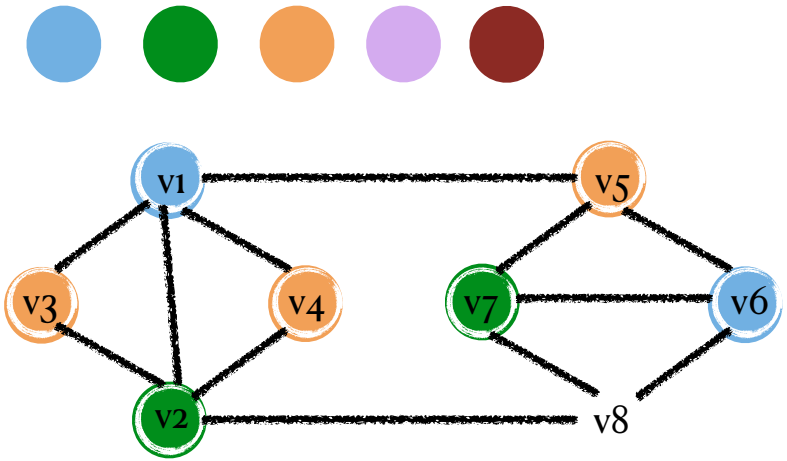
Una Propuesta para el Problema de Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

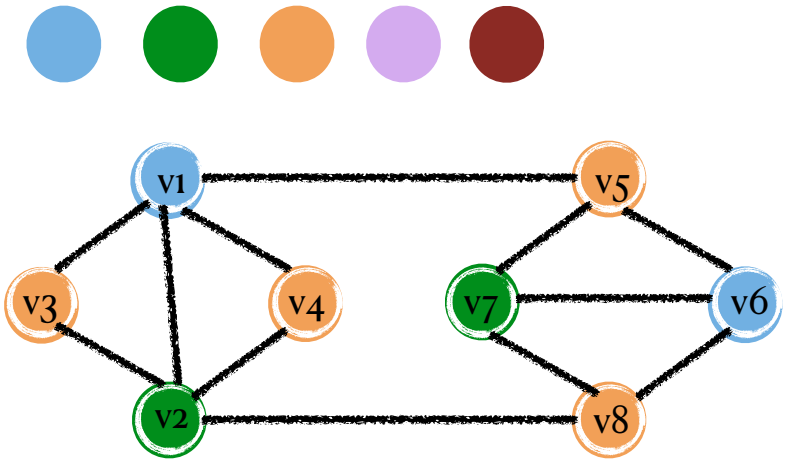
Una Propuesta para el Problema de Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

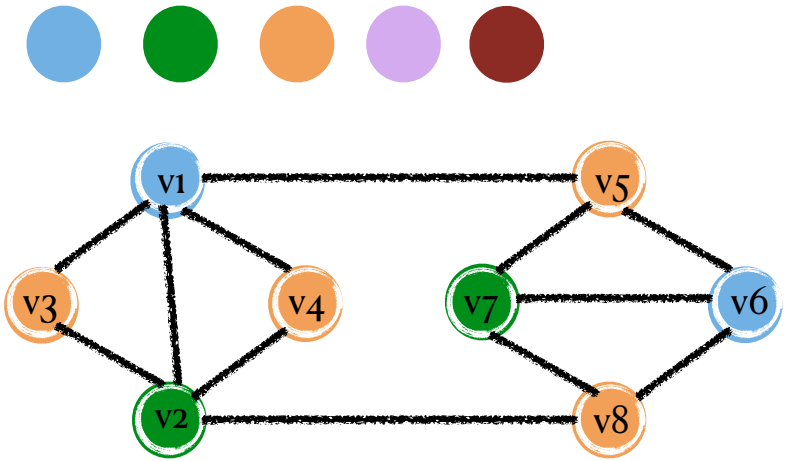
Una Propuesta para el Problema de Coloreo de Grafos



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

Una Propuesta para el Problema de Coloreo de Grafos



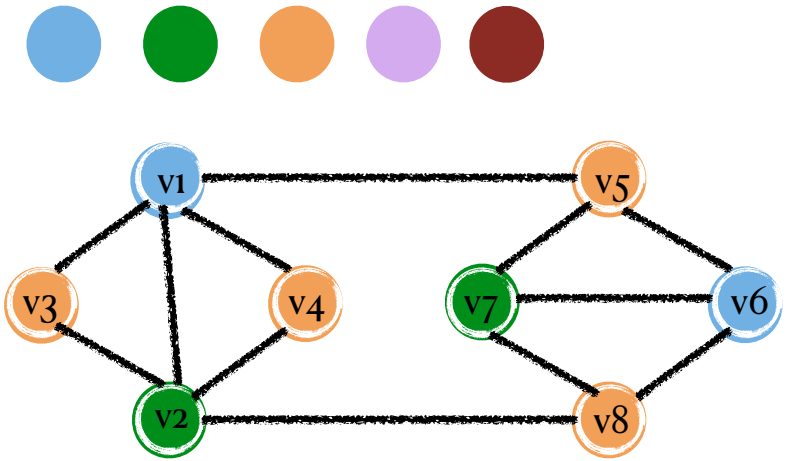
Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

3 colores!

Una Propuesta para el Pro rafos

Dado un orden “correcto”, este algoritmo produce una solución óptima (pero hay que conocer cuál es ese orden)



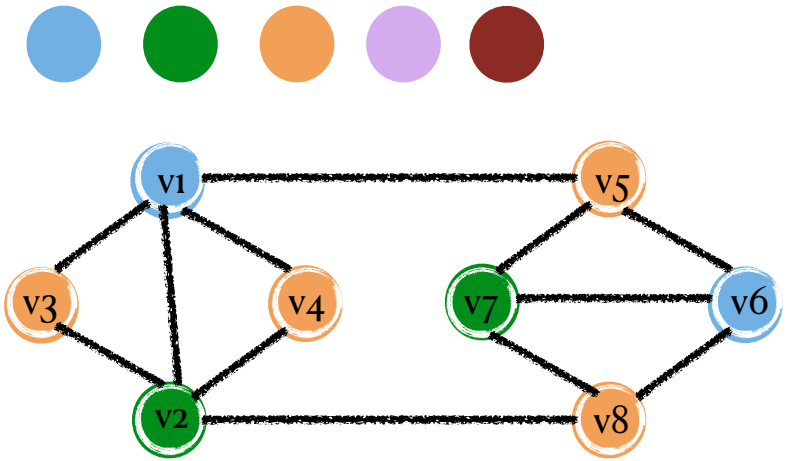
Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

3 colores!

Una Propuesta para el Pro rafos

Dado un orden “correcto”, este algoritmo produce una solución óptima (pero hay que conocer cuál es ese orden)



Orden de visita

[v1, v6, v2, v7, v5, v3, v4, v8]

3 colores!

Colorea de Grafos

Búsqueda Exhaustiva

```
Algoritmo EncontrarNumeroColores(G)
//Encuentra el número de colores  óptimo
//Entrada: El objeto Grafo G
//Salida: El número  mínimo de colores (m) y la asignación final de colores (ColorDe)

n ← G.CantidadVertices();
ColorDe = new array[n];



// Inicializar todos los vértices del grafo con color 0 (sin color)
for i ← 0 to n - 1 do
    v ← G.ObtenerVertice(i);
    ColorDe[v] ← 0;

// Búsqueda del número mínimo de colores.

for m ← 1 to n do
    if Coloreo(G, m, ColorDe, 0) then
       break;
```

Coloreo de Grafos

Búsqueda Exhaustiva

```
Algoritmo Coloreo(G, m, ColorDe, nodoInicio)
//Intenta colorear el grafo avanzando a través de sus vértices por índice
//Entrada: El objeto Grafo G, la cantidad máxima de colores m,  el diccionario ColorDe, y el índice i del vértice actual (empieza en 0)
//Salida: Devuelve verdadero si logra una coloración válida con m colores

if nodoInicio == G.CantidadVertices() then
    return true //Todos los vértices del grafo han sido coloreados con éxito

v ← G.ObtenerVertice(i) //Obtenemos el vértice actual

for c ← 1 to m do
    if EsSeguro(G, v, ColorDe, c) then
        ColorDe[v] ← c //Asignación tentativa

        // Avanzamos al siguiente vértice del grafo usando el índice  nodoInicio + 1
        if Coloreo(G, m, ColorDe, nodoInicio + 1) then
            return true;

        ColorDe[v] ← 0 // Paso de Backtracking (se deshace la asignación si falló)

return false // Ningún color de 1 a m funcionó para este vértice en esta rama
```

Búsqueda Exhaustiva

- El término "búsqueda exhaustiva" también puede ser aplicada a dos importantes algoritmos que sistemáticamente procesan todos los vértices de un grafo.
    - Depth first search
    - Breadth first search

Depth First Search

```
ALGORITHM DFS(G)
// Entrada: Grafo G = ⟨V, E⟩
// Salida: Grafo G con sus vértices marcados con enteros consecutivos
// en el orden en que son encontrados por primera vez durante el recorrido DFS.

// marcar cada vértice de V con 0 como marca de “no visitado”

count ← 0
for each vertex v in V do
  if v is marked with 0
    dfs(v)
```



```
dfs(v)
// visita recursivamente todos los vértices no visitados conectados al vértice v
// mediante un camino y los numera en el orden en que son encontrados
// mediante la variable global count

count ← count + 1
marcar v con count
for each vertex w in  adyacente(v) do
    if w está marcado con 0
        dfs(w)
```

Depth First Search

```
ALGORITHM DFS(G)
// Entrada: Grafo G = ⟨V, E⟩
// Salida: Grafo G con sus vértices marcados con enteros consecutivos
// en el orden en que son encontrados por primera vez durante el recorrido DFS.

// marcar cada vértice de V con 0 como marca de “no visitado”

count ← 0
for each vertex v in V do
  if v is marked with 0
    dfs(v)
```

Función Recursiva

```
dfs(v)
// visita recursivamente todos los vértices no visitados conectados al vértice v
// mediante un camino y los numera en el orden en que son encontrados
// mediante la variable global count

count ← count + 1
marcar v con count
for each vertex w in  adyacente(v) do
    if w está marcado con 0
        dfs(w)
```

Depth First Search

¿Tiempo de Ejecución?

```
ALGORITHM DFS(G)
// Entrada: Grafo G = ⟨V, E⟩
// Salida: Grafo G con sus vértices marcados con enteros consecutivos
// en el orden en que son encontrados por primera vez durante el recorrido DFS.

// marcar cada vértice de V con 0 como marca de “no visitado”

count ← 0
for each vertex v in V do
  if v is marked with 0
    dfs(v)
```

Función Recursiva

```
dfs(v)
// visita recursivamente todos los vértices no visitados conectados al vértice v
// mediante un camino y los numera en el orden en que son encontrados
// mediante la variable global count

count ← count + 1
marcar v con count
for each vertex w in  adyacente(v) do
    if w está marcado con 0
        dfs(w)
```

Depth First Search

¿Tiempo de Ejecución?

Con Listas de adyacencias?
Con Matrices?

```
ALGORITHM DFS(G)
// Entrada: Grafo G = ⟨V, E⟩
// Salida: Grafo G con sus vértices marcados con enteros consecutivos
// en el orden en que son encontrados por primera vez durante el recorrido DFS.

// marcar cada vértice de V con 0 como marca de “no visitado”

count ← 0
for each vertex v in V do
  if v is marked with 0
    dfs(v)
```

Función Recursiva

```
dfs(v)
// visita recursivamente todos los vértices no visitados conectados al vértice v
// mediante un camino y los numera en el orden en que son encontrados
// mediante la variable global count

count ← count + 1
marcar v con count
for each vertex w in  adyacente(v) do
    if w está marcado con 0
        dfs(w)
```

Depth First Search

¿Tiempo de Ejecución?

Con Listas de adyacencias? $O(V+E)$
Con Matrices?

```
ALGORITHM DFS(G)
// Entrada: Grafo G = ⟨V, E⟩
// Salida: Grafo G con sus vértices marcados con enteros consecutivos
// en el orden en que son encontrados por primera vez durante el recorrido DFS.

// marcar cada vértice de V con 0 como marca de “no visitado”

count ← 0
for each vertex v in V do
  if v is marked with 0
    dfs(v)
```

Función Recursiva

```
dfs(v)
// visita recursivamente todos los vértices no visitados conectados al vértice v
// mediante un camino y los numera en el orden en que son encontrados
// mediante la variable global count

count ← count + 1
marcar v con count
for each vertex w in  adyacente(v) do
    if w está marcado con 0
        dfs(w)
```

Depth First Search

¿Tiempo de Ejecución?

Con Listas de adyacencias? $O(V+E)$
Con Matrices? $O(V^2)$

```
ALGORITHM DFS(G)
// Entrada: Grafo G = ⟨V, E⟩
// Salida: Grafo G con sus vértices marcados con enteros consecutivos
// en el orden en que son encontrados por primera vez durante el recorrido DFS.

// marcar cada vértice de V con 0 como marca de “no visitado”

count ← 0
for each vertex v in V do
  if v is marked with 0
    dfs(v)
```

Función Recursiva

```
dfs(v)
// visita recursivamente todos los vértices no visitados conectados al vértice v
// mediante un camino y los numera en el orden en que son encontrados
// mediante la variable global count

count ← count + 1
marcar v con count
for each vertex w in  adyacente(v) do
    if w está marcado con 0
        dfs(w)
```

Breadth First Search

```
ALGORITHM BFS(G)
// Entrada: Grafo G = ⟨V, E⟩
// Salida: Grafo G con sus vértices marcados con números consecutivos en el orden en que son encontrados por
// primera vez por el recorrido BFS

marcar cada vértice en V con 0 como marca de "no visitado"
count ← 0
for each vertex v in V do
  if v is marked with 0
    bfs(v)
```

```
bfs(v)
// visita todos los vértices no visitados conectados al vértice v
// mediante un camino y los numera en el orden en que son
// visitados a través de la variable global count

count ← count + 1; //marcar v con count e inicializar una cola con v
while the queue is not empty do
  for each vertex w in adjacent(front vertex) do
    if w is marked with 0
      count ← count + 1;
      marcar w con count
      agregar w a la cola
    eliminar el vértice del frente de la cola
```

Consideraciones Finales sobre Fuerza Bruta

Los algoritmos de búsqueda exhaustiva poseen en general tiempos de corrida razonables sólo para instancias pequeñas del problema a resolver.

En algunos casos, existen alternativas mucho mejores.

En muchos casos, la búsqueda exhaustiva (o sus variantes) es la única forma conocida de resolver ciertos problemas de manera exacta.